Comment optimiser les communications de Layer 2 au sein de LANs ?

Analyse & Synthèse



FRANÇOIS LEVEQUE | PRINTEMPS 2025

Contexte : 2
Introduction à la Topologie : 2
Fonctionnement de base des réseaux : 2
Trafic Broadcast : 2
Réseau Ethernet Classique : 3
Introduction aux VLANs : 3
Besoins en équipements : 3
Réalisation Concrète : 3
Subdivision en sous-réseaux : 4
Application des plages adresse aux VLANs : 5
Configuration complète des VLANs : 6
Intérêt des réseaux virtuels : 6
Configuration Spécifique et liens Trunk : 7
Routage Inter-VLAN : 9
Essai de connectivité inter-VLAN : 9
Connectivité au-delà du LAN : 11
Redondance de Layer 2 IEEE 802.1D : 12
Pertinence du protocole STP & Notion de trafic Broadcast : 12
Fonctionnement du protocole 802.1D : 13
Premier essai de redondance : 17
Solutions pour accélérer la re convergence dans une topologie 802.1D : 22
Démonstration d'un Broadcast Storm : 26
Conclusion sur le protocole 802.1D : 28
Redondance de Layer 2 IEEE 802.1w : 28
Evolution du protocole IEEE 802.1D : 28
Processus de Synchronisation RSTP : 30
Examen des Logs lors du processus de Synchronisation RSTP : 32
Examen des Trames lors du processus de Synchronisation RSTP : 33
Gestion d'incident d'un domaine par RSTP : 35
Résumé du Rapid Spanning-Tree : 37
Multiple Spanning-Tree : 38
Pertinence du protocole IEEE 802.1s : 38
Fonctionnement de MSTP : 38
Premier essai de fonctionnement : 39
Equilibrage des charges : 41
Synthèse des protocoles Spanning-Tree : 41
Solutions Alternatives : 42

Agrégation de Ports : 42
Agrégation de Ports couplée à MSTP : 42
Architecture de LANs : 44
Fabrique : 44
Architecture Spine-Leaf : 45
Application Pratique : 46
Conclusion Générale : 46

Contexte :

Ce projet s'inscrit dans une démarche de compréhension avancée des LANs et du fonctionnement des protocoles de Layer 2. Différents procédés de gestion de redondance de Layer 2 seront étudiés et comparés dans le but d'établir des solutions efficaces adaptées à différentes situations.

L'objectif second de ce projet est la préparation à certains aspects de la certifications Cisco CCNP. Notamment les notions 3.1.a et 3.1.c de la liste des éléments à étudier pour cette certification.

Introduction à la Topologie :

Fonctionnement de base des réseaux :

Chaque interface réseau d'un équipement possède deux types d'adresse ; une adresse MAC (6 octets) et une adresse IP (4 octets). Deux adressages différents sont utilisés, car en réseau, le comportement de l'émetteur est différent selon la proximité du destinataire. L'émetteur doit être capable de différencier si le destinataire est à portée physique – de câblage – ou si les fonctions du réseau, fonctions de routage, doivent être sollicitées pour acheminer le trafic. Les adresses MAC sont utilisées au sein des LAN (réseaux locaux), lorsque les machines sont à portée de câblage où le routage n'est pas utilisé. On appelle ce type de trafic, le trafic de Layer 2, l'unité de transmission se nomme la Trame Ethernet. Cette dernière est composée de différents champs comme les adresse MAC de l'émetteur et du récepteur.

Tandis que les adresses IP sont utilisées pour acheminer le trafic à travers des réseaux différents qui ne sont pas à portée de câblage.

Trafic Broadcast :

Le trafic Broadcast est un type de trafic destiné à tous les hôtes d'un réseau / sous-réseau, il utilise l'adresse MAC : FF:FF:FF:FF:FF:FF. Ce type de trafic est essentiel au fonctionnement des réseaux car il permet de résoudre une problématique majeure : Que faire quand le Switch n'a pas d'entrée pour la destination dans sa table d'adresse MAC ?

Nous sommes dans le cas de l'œuf et la poule ; pour communiquer avec un ordinateur je dois avoir son adresse, mais pour avoir son adresse je dois communiquer avec lui.

Le trafic Broadcast résout cette problématique et permet d'obtenir les adresses des autres ordinateurs sur le réseau en envoyant des messages à destination de tous. Son fonctionnement sera explicité plus tard, voir Pertinence du protocole STP & Notion de trafic Broadcast :

En somme, son utilisation est indispensable au fonctionnement des réseaux.

Réseau Ethernet Classique :

Dans un réseau Ethernet classique, sans VLAN, **le domaine de broadcast** (c'est-à-dire l'ensemble des machines qui reçoivent un message de type broadcast) est **défini par le câblage physique** : toutes les machines connectées à un même switch appartiennent au même domaine de broadcast.

Cela pose un problème : **ce critère est uniquement géographique**. On regroupe les machines parce qu'elles sont proches physiquement, pas parce qu'elles ont une fonction commune. Or, en entreprise, on préférerait souvent regrouper les équipements **par fonction ou service**.

Introduction aux VLANs :

Les VLANs sont des réseaux locaux virtuels, ils permettent de regrouper différentes machines dans le même domaine de broadcast même si ces machines ne sont pas proches géographiquement. Un VLAN répond à la problématique de proximité et affranchit le réseau de difficultés liées au câblage. Pour regrouper des machines il n'est plus nécessaire que celles-ci soient connectées au même Switch, plus besoin de tirer les câbles, il suffit de les assigner virtuellement au même VLAN. Cette technologie étend le support de communication sur plusieurs Switches.

Besoins en équipements :

Le projet met en jeu l'utilisation d'un réseau complexe, doté d'une capacité de gestion de trafic importante. Il est nécessaire de s'assurer de la redondance du réseau dans le but d'anticiper d'éventuelles pannes. Cela signifie que le trafic doit avoir la capacité d'emprunter différents chemins en fonction de la disponibilité des liens, d'autres équipements doivent être également disponibles pour remplacer les défaillances. En aucun cas la transmission de données ne doit se voir durablement impactée par un problème d'infrastructure.

En somme, la topologie doit respecter les contraintes citées précédemment, ce qui implique un besoin matériel plus important et une complexification du paramétrage des appareils utilisés.

Réalisation Concrète :

La topologie serait composée de quatre Switches de Layer 3, avec une capacité de routage dans le but de diriger, sans l'aide d'un routeur, le trafic inter-VLAN. Trois VLANs différents seront également utilisés au sein du réseau afin de segmenter la topologie au Layer 2 et de simuler des conditions réelles. Ainsi, comme illustré *fig. 1.1*, la topologie est composée de quatre Switches et d'un routeur, associés à des hôtes utilisant une exploitation linux. Les hôtes seront majoritairement utilisés pour effectuer des requêtes *Ping* et *Traceroute* pour s'assurer de la bonne connectivité des appareils au sein du réseau.

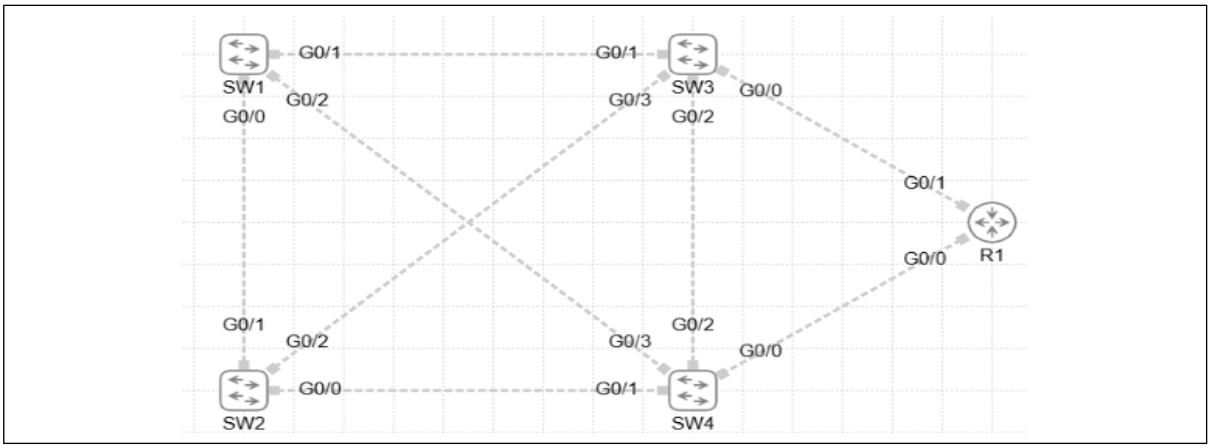


Figure 1.1 : Premier aperçu de la topologie

Le concept de redondance est déjà appliqué, chaque composante du réseau admet au moins deux chemins distincts vers une autre composante, assurant efficience et rapidité de convergence après un incident.

Subdivision en sous-réseaux :

Nous pouvons d'ores et déjà anticiper l'organisation des trois réseaux virtuels locaux. Par souci de simplicité, le réseau global utilise une adresse privée de classe C. Soit l'adresse suivante : 192.168.1.0/24

Rappelons qu'une adresse IP est composée de quatre octets, soit 32 bits.

Le masque de sous-réseau /24 se caractérise par trois octets réservés pour la partie réseau et un dernier octet dédié aux adresses de hôtes. On en conclut que ce masque permet de réserver 256 adresses, incluant l'adresse *Broadcast* et *Réseau*.

Tableau 1.1 : Démonstration d'un masque /24

Masque	Partie Adresse Réseau	Partie Adresses Hôtes
/24	11111111.11111111.11111111	.00000000

Il est nécessaire de diviser cette adresse afin d'obtenir les plages *IP* des sous-réseaux, pour cela, 2 bits du dernier octet doivent être « empruntés » à la partie hôte pour étendre le masque de 2 bits, soit un masque /26.

Tableau 1.2 : Passage à un masque /26 visant un accueil de sous-réseaux

Masque	Partie Adresse Réseau	Partie Adresses Hôtes
/26	11111111.11111111.11111111	.11000000

En empruntant deux bits, nous avons désormais quatre sous-réseaux de masque /26, puisque $2^2 = 4$.

Nous devons déterminer les plages et adresses de ces sous-réseaux. Il nous reste 6 bits dans la partie Hôtes, soit $2^6 = 64$ adresses par sous-réseau.

Tableau 1.3 : Découpage des plages adresse par sous-réseau

Sous-Réseau	Adresse Réseau	Plage utilisable	Adresse Broadcast
1.	192.168.1.0/26	192.168.1.1 - 62	192.168.1.63
2.	192.168.1.64/26	192.168.1.65 - 126	192.168.1.127
3.	192.168.1.128/26	192.168.1.129 - 190	192.168.1.191
4.	192.168.1.192/26	192.168.1.193 - 254	192.168.1.255

Application des plages adresse aux VLANs :

Nous pouvons maintenant assigner chaque adresse aux différents VLANs comme suit :

Tableau 1.4 : Association d'adresses réseau aux VLANs suivant leur numéro d'identification

VLAN n°	Adresse
VLAN 10	192.168.1.64/26
VLAN 20	192.168.1.128/26
VLAN 30	192.168.1.192/26

L'évolution de la topologie est disponible *fig. 1.2*.

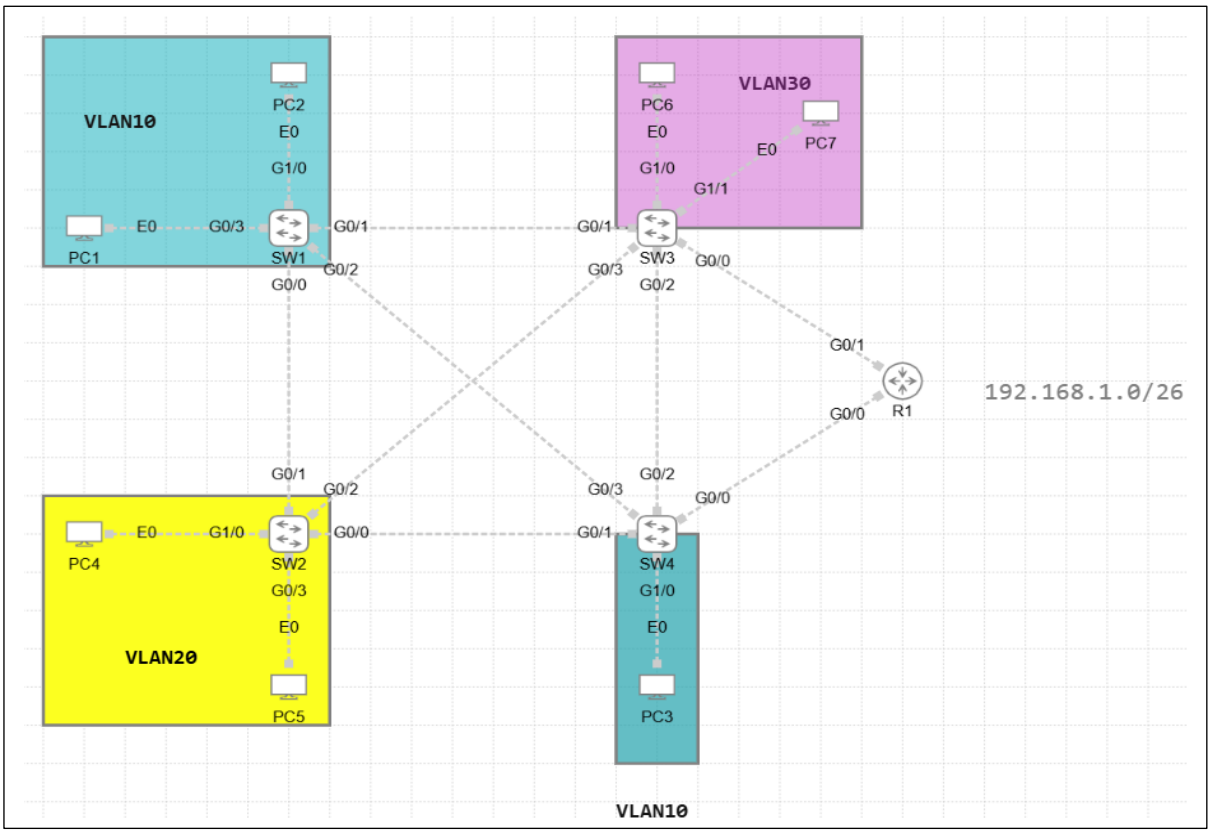


Figure 1.2 : Topologie Finale et segmentation de Layer 2

Cette topologie sera utilisée dans toutes les sections du projet, notamment dans les sections sur le Spanning-Tree où il sera nécessaire de s'y référer afin de comprendre les expériences et simulations de pannes notamment.

Configuration complète des VLANs :

Intérêt des réseaux virtuels :

Comme énoncé, le réseau accueille trois VLANs distincts, ce dispositif de réseaux virtuels permet de segmenter la topologie au Layer 2, subdivisant les domaines de Broadcast. Pour comprendre l'avantage de cette séparation, il est essentiel de rappeler le fonctionnement du Layer 2.

Prenons l'exemple d'un cas classique où un switch reçoit une Trame (Frame : *PDU de Layer 2)*, il s'agit d'une Trame de type *Unknown unicast*, c'est-à-dire que le switch n'a pas d'entrée dans sa table d'adresse MAC pour cette trame, *voir schéma de table MAC*.

Chaque switch construit une table d'adresses MAC, comme illustré ci-dessous, et s'en sert pour diriger le trafic au sein du LAN.

Cette table est construite après réception de trafic, *exemple :* PC1 envoie du trafic sur l'interface G0/1 du Switch, alors l'entrée G0/1 de la table sera associée à l'adresse MAC de PC1.

Tableau 1.5 : Représentation d'une table d'adresses MAC selon un Switch

Adresse MAC	Interface	Statut
A1:B2:C3:D4:E5	G0/1	Connu
B2:C3:D4:E5:F6	G0/2	Connu
C3:D4:E5:F6:A1	?	Inconnu

Pour revenir à l'exemple, si le Switch reçoit du trafic destiné à l'adresse C3:D4:E5:F6:A1, il n'y a pas d'entrée dans la table pour cette adresse, le Switch va donc relayer le trafic sur tous ses ports exceptés celui à l'origine de la Trame. Toutes les composantes qui reçoivent la Trame vont examiner l'adresse de destination, si cette adresse correspond avec la leur elles vont traiter la Trame, sinon elles l'ignoreront. Enfin, le Switch va ajouter une entrée dans sa table d'adresse MAC pour la nouvelle adresse.

Le cas de Trame de type *Unknown Unicast* est récurrent au sein d'un LAN, nous comprenons alors l'intérêt de segmenter le réseau global afin de limiter la propagation d'envois massifs de Trames depuis toutes les interfaces. De plus, si plusieurs Switches sont utilisés, chaque appareil relaiera la Trame depuis toutes ses interfaces, *voir tableau 1.6*.

Tableau 1.6 : Synthèse de gestion de type de trafic de Layer 2

Type de trafic	Descriptif	Action
Unknown Unicast	Aucune entrée dans la table MAC, unique destinataire	Relayer la Trame sur tous les ports, excepté celui d'origine de la Trame
Known Unicast	Entrée correspondante dans la table MAC, unique destinataire	Envoyer la trame par le biais du port correspondant de la table MAC

Configuration Spécifique et liens Trunk :

Chaque VLAN doit être configuré sur tous les Switches, de plus des liens de type *Trunk* doivent être configurés entre chaque Switch pour accueillir le trafic des différents VLANs qui composent le réseau, cela permet aux différents VLANs de communiquer sur un même lien.

Le trafic doit être identifié comme appartenant à tel ou tel VLAN, pour cela nous utilisons le standard ***IEEE 802.1Q*** [1]qui permet d'identifier le trafic en l'associant à un VLAN avec un *tag*. La section *Vlan ID (VID)* est utilisée pour identifier le trafic, *voir structure du tag 802.1Q*.

Ce tag est ajouté dans la Trame Ethernet classique incrémentant sa taille initiale de 4 octets, *voir* tableau ci-dessous

Tableau 1.7 : Structure du tag 802.1Q & Insertion dans une Trame Ethernet simplifiée

16 bits	3 bits	1 bit	12 bits
TPID	TCI		
	PCP	DEI	VID

Adresse MAC dst	Adresse MAC src	802.1Q Tag	Taille de la trame/EtherType	Data	Frame Check Sequence (FCS)

La configuration débute en associant les interfaces désirées de chaque switch à un VLAN choisi. On assigne ici des interfaces de Switch à des VLANs, alors, chaque hôte branché sur une de ces interfaces sera lui-même identifié comme faisant parti du VLAN choisi.

[1] La norme 802.1Q permet de lier le trafic à un VLAN en utilisant un système de tag. Cette norme vise à modifier la trame Ethernet au niveau de la couche MAC afin de fournir un système d'étiquetage, voir https://fr.wikipedia.org/wiki/IEEE_802.1Q

SW (config)# *interface [interface]*

SW (config-if)# *switchport mode access*

SW (config-if)# *switchport access vlan [vlan_id]*

SW# *show vlan brief*

Voir exemple de configuration fig. 2.2.

```
SW1(config-if)#int g0/3
SW1(config-if)#sw mod acc
SW1(config-if)#sw acc vlan 10
SW1(config-if)#exit
SW1(config)#do sh vlan br

VLAN Name                              Status    Ports
---- --------------------------------- --------- -------------------------------
1    default                           active    Gi0/0, Gi0/1, Gi0/2, Gi1/1
                                                 Gi1/2, Gi1/3, Gi2/0, Gi2/1
                                                 Gi2/2, Gi2/3
10   VLAN0010                          active    Gi0/3, Gi1/0
1002 fddi-default                      act/unsup
1003 token-ring-default                act/unsup
1004 fddinet-default                   act/unsup
1005 trnet-default                     act/unsup
SW1(config)#
```

Figure 2.2 : Configuration du VLAN10 sur SW1

Comme énoncé, les liens de type *Trunk* doivent être configurés sur chaque liaison Switch <-> Switch. Ceux-ci seront configurés de manière statique comme observable *fig. 2.3*.

```
SW2(config)#int g0/1
SW2(config-if)#sw trunk encap dot1
SW2(config-if)#sw mode trunk
SW2(config-if)#sw trunk allowed vlan 10,20,30
SW2(config-if)#do sh int trunk

Port        Mode             Encapsulation  Status        Native vlan
Gi0/1       on               802.1q         trunking      1

Port        Vlans allowed on trunk
Gi0/1       10,20,30

Port        Vlans allowed and active in management domain
Gi0/1       20

Port        Vlans in spanning tree forwarding state and not pruned
Gi0/1       20
SW2(config-if)#
```

Figure 2.3 : Configuration d'un lien Trunk entre deux Switch

Tous les VLANs ont été créés et associés aux Switches suivant la topologie *fig.1.2*, il est désormais nécessaire d'implanter une solution de routage inter-VLANs. En effet, les VLANs étant dans différents sous-réseaux – avec des adresses différentes – les communications inter-VLANs doivent être routées.

Routage Inter-VLAN :

Plusieurs solutions sont envisageables pour répondre à cette problématique. Il est possible de citer *Router-on-a-Stick (ROAS)* qui consiste à créer des *sous-interfaces* sur un routeur qui se chargerait d'orienter le trafic de Layer 3 au sein du LAN. Cette solution est simple à mettre en place mais se révèle inefficace pour des réseaux comportant un nombre important de VLANs. De plus, le trafic de Layer 3 est significativement plus complexe à traiter que celui de Layer 2. Nous avons tout intérêt à simplifier au maximum les communications inter-VLANs par souci d'efficience.

La solution retenue est d'exclure le routeur des transmissions inter-VLANs, de fait, les Switches modernes supportent le routage de Paquets. Suivant le schéma de la topologie, les Switches 3 et 4 seront utilisés pour router le trafic entre les sous-réseaux virtuels. Pour ce faire, des *Switch Virtual Interfaces (SVIs)* seront utilisées.

Ces *SVIs* permettent aux Switches de se comporter comme des Routeurs. Une SVI doit être configurée par VLAN et par Switch sur ceux connectés au Routeur pour gérer le trafic inter-VLANs. La création de plusieurs SVIs par VLAN assure la redondance voulue des liens au sein du LAN, à condition d'utiliser un *First Hop Redundancy Protocol (FHRP)* qui permet d'utiliser deux passerelles par défaut. Le concept de *FHRP* sera expliqué plus tard.

Le dédoublement de SVIs sur des switches différents est inutile sans protocole comme *HSRP* en plus, car un hôte ne peut utiliser qu'une seule passerelle par défaut.

La configuration de ces interfaces virtuelles s'effectue par VLAN comme tel :

SW(config)# *interface vlan [vlan_id]*

SW(config-if)# *ip address [ip] [mask]*

SW(config-if)# *no shut*

```
SW3>en
SW3#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
SW3(config)#int vlan 30
SW3(config-if)#
*Jan 28 11:29:01.462: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan30, changed state to down
SW3(config-if)#ip add 192.168.1.193 255.255.255.192
SW3(config-if)#no sh
SW3(config-if)#
*Jan 28 11:30:21.097: %LINK-3-UPDOWN: Interface Vlan30, changed state to up
*Jan 28 11:30:22.097: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan30, changed state to up
```

Figure 2.4 : Configuration de la SVI de VLAN 30 avec l'adresse 192.168.1.193/26

Essai de connectivité inter-VLAN :

Une fois tous les liens configurés, nous pouvons tester la connectivité entre les différentes composantes du réseau tout en examinant les paquets afin de vérifier le bon fonctionnement des *SVIs* et liens *Trunks*.

Pour tester la connectivité ; des requêtes *ICMP* seront envoyées à partir de *PC1* à l'adresse 192.168.1.68 dans le VLAN 10, vers *PC4* à l'adresse 192.168.1.30 dans le VLAN 20. Le lien entre SW1 et SW3 sera volontairement désactivé afin de forcer le trafic à suivre le chemin suivant : PC1 -> SW1 -> SW4 -> SW2 -> PC4.

Nous observons une réponse de PC4, ce qui signifie que la connectivité fonctionne correctement.

Examinons d'abord le trafic entre SW1 et SW4 :

	Time	Source	Destination	Protocol	Lengtl	Info
4	1.534051	192.168.1.68	192.168.1.130	ICMP	102	Echo (ping) request
5	1.591836	192.168.1.130	192.168.1.68	ICMP	102	Echo (ping) reply

Figure 2.5 : ICMP Echo Request / Reply entre PC1 (VLAN10) et PC4 (VLAN20)

On observe la requête et la réponse ICMP entre les deux machines. En s'attachant au Paquet 4, nous constatons que le lien *Trunk* fonctionne correctement, le trafic est relié au VLAN 10, comme observable sur la *figure 2.6* dans l'onglet 802.1Q. De plus, les adresses de départ et d'arrivée correspondent à celles configurées sur les hôtes des différents VLANs.

```
> Frame 4: 102 bytes on wire (816 bits), 102 bytes captured (816 bits)
> Ethernet II, Src: 52:54:00:05:4a:ad (52:54:00:05:4a:ad), Dst: 52:54:00:0b:80:0a (52:54:00:0b:80:0a)
v 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 10
    000. .... .... .... = Priority: Best Effort (default) (0)
    ...0 .... .... .... = DEI: Ineligible
    .... 0000 0000 1010 = ID: 10
    Type: IPv4 (0x0800)
> Internet Protocol Version 4, Src: 192.168.1.68, Dst: 192.168.1.130
> Internet Control Message Protocol
```

Figure 2.6 : Capture de trafic entre SW1 et SW4 après une requête ICMP de PC1

L'interface de routage virtuelle étant configurée sur SW4, le Switch va orienter le trafic en changeant le tag ; passant de 10 (VLAN d'origine) à 20 (VLAN de destination). Il est possible de vérifier ce comportement en examinant le trafic entre SW4 et SW2, *fig. 2.7*.

Wireshark · Paquet 6 · ping_to_trace_VID_2.pcap

```
> Frame 6: 102 bytes on wire (816 bits), 102 bytes captured (816 bits)
> Ethernet II, Src: 52:54:00:0b:80:14 (52:54:00:0b:80:14), Dst: 52:54:00:1e:6e:f6 (52:54:00:1e:6e:f6)
v 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 20
    000. .... .... .... = Priority: Best Effort (default) (0)
    ...0 .... .... .... = DEI: Ineligible
    .... 0000 0001 0100 = ID: 20
    Type: IPv4 (0x0800)
> Internet Protocol Version 4, Src: 192.168.1.68, Dst: 192.168.1.130
```

Figure 2.7 : Capture de trafic entre SW2 et SW4 après une requête ICMP de PC1

Nous constatons que le tag a changé, pointant maintenant vers le VLAN de destination ; VLAN 20.

Connectivité au-delà du LAN :

Afin de s'approcher au maximum d'un scénario réaliste, les hôtes doivent être capables d'envoyer et de recevoir du trafic au-delà du LAN par le biais du Routeur. Il est nécessaire de configurer les liaisons *Point-to-Point* entre le Routeur et les deux Switches pour y parvenir. Suivant la topologie finale *fig. 1.2,* deux liaisons seront configurées entre les appareils. De plus, des *Routes* seront créées, nécessaires à l'acheminement du trafic. Il est possible de configurer manuellement toutes les Routes, or cela n'est pas pertinent à grande échelle.

Nous allons plutôt utiliser le protocole de Routage *Open-Shortest-Path-First (OSPF)* qui calcule les meilleurs Routes d'après l'algorithme de Dijkstra.

L'utilisation de ce protocole permet un échange automatique des Routes entre les composantes de Layer 3 du réseau après la formation d'une liaison OSPF *Full*. De plus, les changements de la topologie seront automatiquement pris en compte sans besoin de reconfiguration manuelle.

Ce protocole met en jeu des zones, appelées *OSPF Areas*, une zone comporte une table commune où les Routes sont stockées, il s'agit d'une *Link State Database (LSDB)* qui comporte les Routes sous forme de *Link State Advertisements (LSAs)*. La topologie utilise une seule zone, l'*Area 0*, autrement appelée *Backbone Area*, *voir schéma 2.8*.

Trois étapes fondamentales composent ce protocole :

1. Formation de liens OSPF Full avec autres routeurs connectés au même segment
2. Echange de *LSAs* avec Routeurs voisins
3. Calcul des meilleurs Routes pour chaque destination et ajout dans la table de Routage

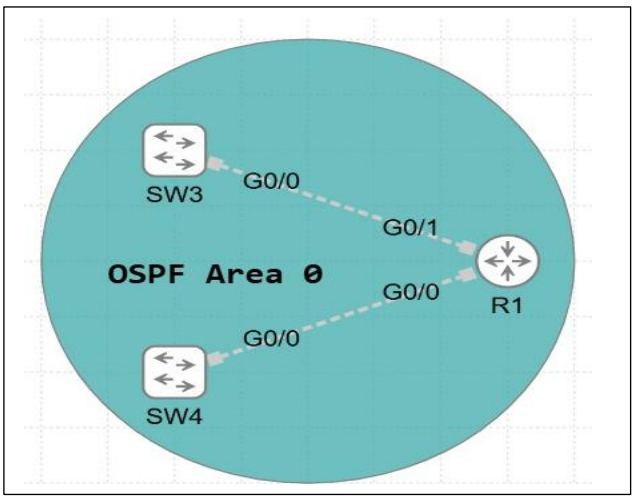


Figure 2.8 : Représentation de la zone OSPF 0

Ce protocole de Routage s'utilise, à l'échelle d'un routeur, comme tel :

R1(config)# *router ospf [ospf_process_id]*

R1(config-router)# *network [adresse_réseau] [wildcard masque] area [numéro_de_zone]*

R1(config-router)# *exit*

L'instruction *network* s'utilise pour chaque adresse réseau qui doit être échangée en tant que *LSA*. De plus, le masque *Wildcard* fait référence au masque de sous-réseau dont tous les bits seraient inversés.

Masque de sous-réseau	Masque Wildcard
255.255.255.0 : /24	**0.0.0.255 : /24**

Une fois cette configuration faite sur les deux Switches et Routeur, nous pouvons examiner la table de routage de R1 afin de s'assurer du bon fonctionnement du protocole.

```
R1#sh ip route | include O
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
       o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
O        192.168.1.64/26 [110/2] via 192.168.1.2, 00:06:52, GigabitEthernet0/1
O        192.168.1.128/26
O        192.168.1.192/26
```

Figure 2.9 : Table de Routage de R1 avec routes OSPF reçues

Toutes les Routes ont été échangées entre les composantes de Layer 3 du réseau, la connectivité est complète.

Redondance de Layer 2 IEEE 802.1D :

Pertinence du protocole STP & Notion de trafic Broadcast :

Comme expliqué, le trafic Broadcast est un type de trafic destiné à tous les hôtes d'un réseau / sous-réseau ou VLAN. Ce trafic est envoyé à la dernière adresse du VLAN appelée adresse Broadcast. Le trafic destiné à cette adresse est relayé par le biais de toutes les interfaces de chaque Switch. D'après la logique de fonctionnement de ce type de trafic ; imaginons l'envoi de trafic Broadcast comme des requêtes DHCP par un client du VLAN, ces requêtes seront identifiées comme destinées à tous les hôtes par le Switch qui relayera le trafic par toutes ses interfaces exceptées celle d'où provient le trafic.

A première vue, cette situation ne semble pas problématique. Or, si plusieurs Switches sont connectés *via* les mêmes segments – situation assez probable – le trafic Broadcast est condamné à se multiplier sans s'arrêter, comme décrit par l'explication ci-dessous.

Ainsi, l'utilisation de trafic broadcast est essentielle au fonctionnement des réseaux mais ne peut exister dans une topologie avec des boucles.

Dans un LAN avec des liens redondants l'envoi de Trames de type *BUM* (Broadcast, Unknown Unicast, Multicast) peut causer des boucles de trafic. En effet, contrairement au Paquet IP, une Trame n'a pas de champ *Time-to-Live (TTL)* qui assure le rejet de Paquets après un certain nombre de sauts (passage d'un routeur). Il est donc virtuellement envisageable qu'une Trame circule en boucle infinie dans le LAN.

L'accumulation de cette situation avec différentes Trames peut causer un *Broadcast Storm*, où l'envoi répété de Trames sape les ressources du réseau, empêchant le trafic habituel de circuler, *voir fig. 3.1*. De plus, à chaque réception d'une Trame Broadcast, cette dernière est renvoyée massivement, multipliant le trafic à chaque occurrence.

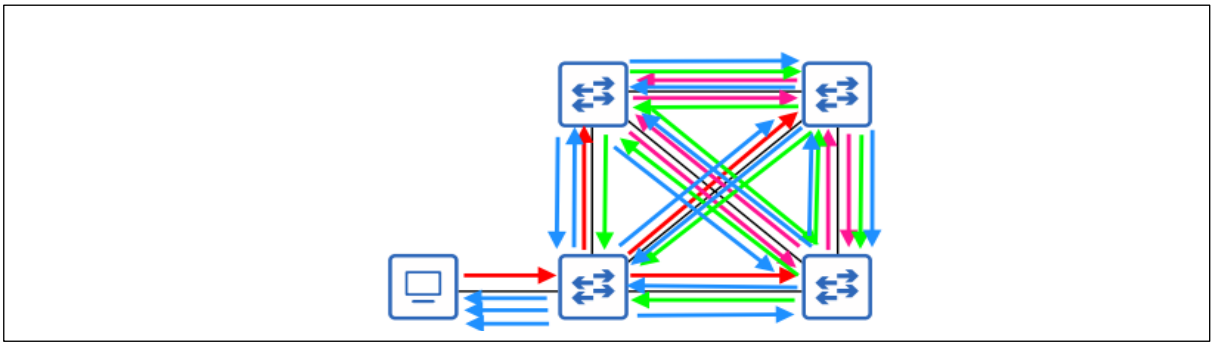


Figure 3.1 : Illustration du concept de Broadcast Storm au sein d'un LAN

Ainsi, il semble nécessaire d'empêcher ce phénomène pour garantir l'efficacité et le bon fonctionnement des LANs. Il faudrait s'assurer qu'il n'y ait pas de boucles dans la topologie, que tous les liens redondants soient désactivés.

Nous cherchons à fabriquer des espaces Broadcast sans boucles, avec un seul chemin vers chaque nœud de la topologie, des espaces sans redondance car cette dernière provoque le souci de tempête broadcast.

Fonctionnement du protocole 802.1D :

Pour cela, le protocole *Spanning-Tree (STP),* IEEE 802.1D est utilisé. Ce dernier permet de désactiver les liens redondants, empêchant les envois de Trames infinis. Nous pouvons alors distinguer la topologie physique de la topologie logique, *fig. 3.2*.

Le résultat de cet algorithme est l'obtention d'une topologie sans lien redondant avec exactement un seul chemin entre chaque nœud.

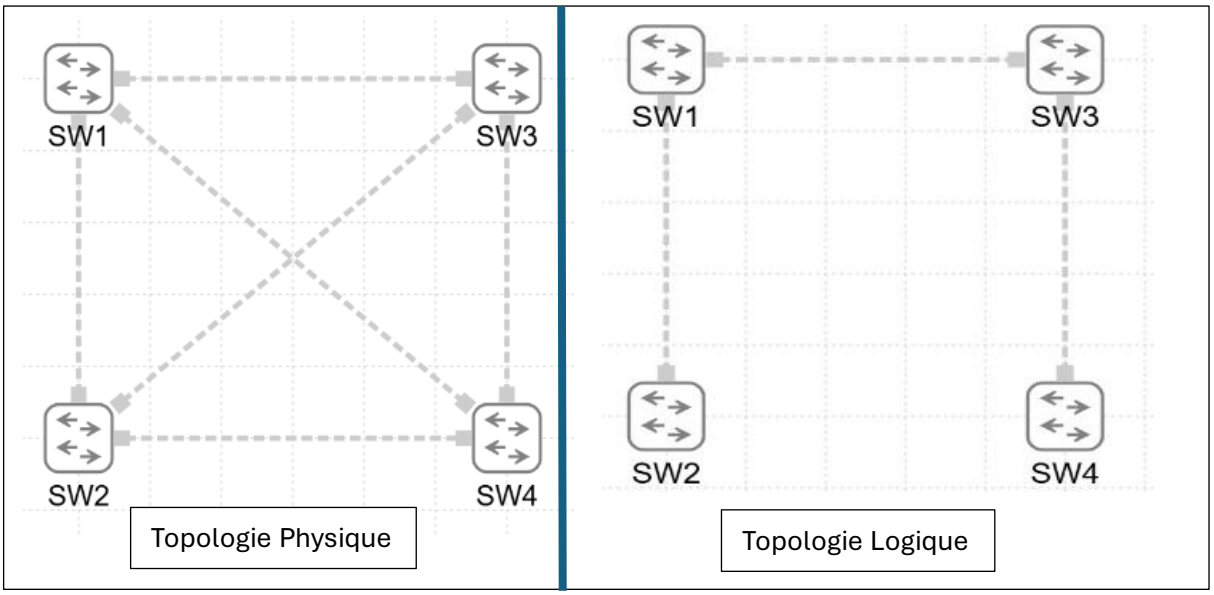


Figure 3.2 : Distinction Logique / Physique suivant le protocole 802.1D

Les liens physiques sont fonctionnels et utilisables, seulement mis, administrativement hors-service par *STP*. Le protocole agit directement sur les interfaces en les plaçant dans différents états : Forwarding ou Blocking.

Tableau 3.1 : Rôles et Etats des ports selon la norme 802.1D[2]

Rôle STP	Etat STP Stable	Fonction	Relaye le trafic
Port Root	Forwarding	Meilleur chemin vers le Root Bridge	OUI
Port Désigné (Designated)	Forwarding	Meilleur chemin d'un segment	OUI
Port Non-Désigné (Non-Designated)	Blocking	Placé en Standby, préviens les boucles sur un segment	NON

Le protocole fonctionne après élection d'un *Root Bridge*, il s'agit d'un Switch servant de point de référence au sein de la topologie.

Pour son fonctionnement, *STP* utilise des *Bridge Protocol Data Units* (*BPDUs*) afin d'élire le Root Bridge, de transmettre des informations et de détecter les boucles.

L'élection du Root Bridge est un processus complexe s'effectuant suivant différents critères, un identifiant de 64 bits est attribué à chaque Switch, il s'agit du *Bridge ID* (*BID*) qui est composé d'une priorité définie – 32768 par défaut – et de l'adresse MAC globale du Switch. Le Switch ayant le plus petit *BID* au sens mathématiques est élu Root Bridge pour la topologie.

Tableau 3.2 : Composantes du Bridge ID (BID)

Bridge ID	
Bridge priority (16 bits)	Adresse MAC (48 bits)

Il est possible de modifier volontairement la topologie *STP* notamment en changeant manuellement la priorité du Switch (Bridge priority), cette dernière se modifie uniquement par incrément de 4096. En théorie, plus la priorité est faible plus les chances d'être élu Root Bridge pour la topologie sont élevés.

Notons qu'à chaque démarrage ou lors d'une perte de connexion avec le Root Bridge un switch s'auto proclamera Root Bridge jusqu'à réception d'un BPDU inférieur à son *BID* (appelé BPDU supérieur). Chaque BPDU envoyé par un Switch X contient le BID du Switch X.

De plus, une fois le *Root Bridge* élu, ce Switch sera le seul à envoyer des BPDUs aux autres Switches, ces derniers ne feront que relayer les BPDUs reçus.

Une fois le Root Bridge élu, chaque port de chaque Switch se verra attribué un rôle associé à un état comme démontré par le tableau ci-dessus. Tous les ports du Root Bridge seront placés dans l'état *Forwarding* avec le rôle de port désigné (*designated port)*.

Ensuite, chaque switch élira un port avec le rôle *Root*, il s'agit de l'interface qui représente le meilleur chemin pour atteindre le *Root Bridge*. Pour cela, la notion de coût est introduite par le protocole selon la vitesse de transmission des interfaces comme tel :

[2] https://networklessons.com/spanning-tree/spanning-tree-port-states

Tableau 3.3 : Coût STP suivant la vitesse du lien

Vitesse	Coût STP
10 Mbps	100
100 Mbps	19
1 Gbps	4
10 Gbps	2

Plus le coût est élevé, moins l'interface sera priorisée en tant que *Root Port*. En cas d'égalité d'autres critères sont utilisés pour trancher :

1. Plus petit Coût Root : la somme des coûts pour atteindre le *Root Bridge*
2. Plus petit *BID* d'un Switch Voisin
3. Plus petit identifiant d'interface d'un Switch voisin, *exemple G0/0 < G0/1*
4. Plus petit identifiant d'interface locale, *exemple G0/0 < G0/1*

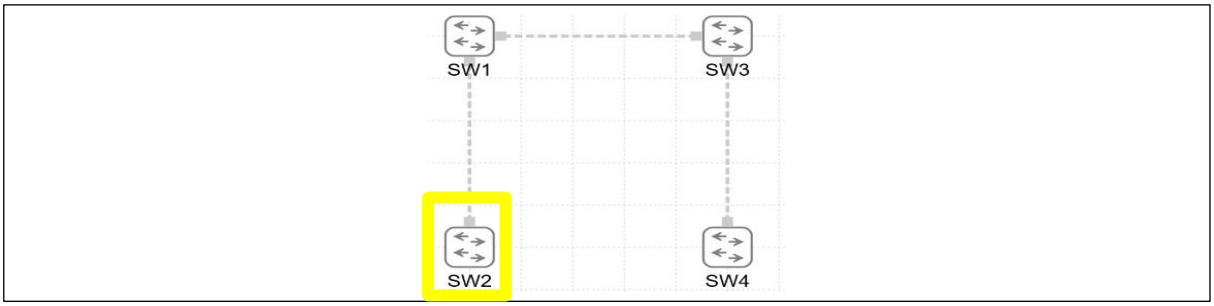


Figure 3.3 : Exemple de calcul de coût Root

Par exemple, *fig.3.3,* pour atteindre le *Root Bridge* (SW2), le coût pour SW3 *via* le seul chemin disponible est de 8, car, sont comptabilisés : le coût de SW3 = 4 et le coût de SW1 = 4. Rappelons que la somme pour atteindre le *Root Bridge* est comptabilisée et non le coût unitaire de chaque interface.

Deux rôles doivent encore être distribués, le choix des interfaces désignées s'effectue dans l'ordre comme présenté :

1. Plus petit coût Root
2. Plus petit *BID*
3. Plus petit identifiant d'interface locale

Il n'existe qu'une seule interface désignée par segment, tous les critères de comparaison s'effectuent en compétition entre deux Switches qui composent un même segment. Toutes les autres interfaces adoptent le rôle de *non-designated*.

En synthèse, les ports *Roots* vont de pair avec les ports designated et les ports designated sont associés aux ports non-designated, *fig.3.4*.

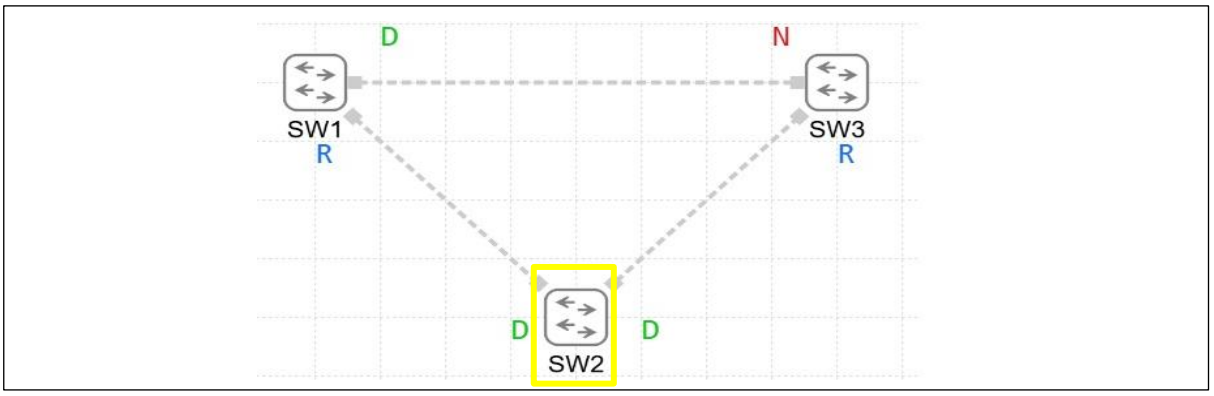


Figure 3.4 : Attribution de rôles suivant le protocole STP

Ici, SW2 est élu Root Bridge, toutes ses interfaces adoptent le rôle de designated, ensuite, un port Root est élu pour chaque Switch puis vient l'élection de l'interface designated. Nous constatons que le lien physique entre SW3 et SW4 est mis, administrativement, hors-service et son évolution dépendra d'une re convergence du domaine STP.

Ainsi, le protocole STP relève d'un système complexe, essentiel au bon fonctionnement des LANs. Néanmoins, les Switches Cisco utilisent une version propriétaire, dérivée de STP qui se nomme *Per-Vlan-Spanning-Tree* + (*PVST*+). Ce protocole fonctionne sensiblement de la même manière que STP classique. La divergence majeure étant que PVST+ utilise une instance STP par VLAN plutôt qu'une instance qui engloberait tout le LAN comme STP.

L'attribution des rôles demeure inchangée et les mécanismes de convergence sont les mêmes.

La *figure 3.5* se charge d'illustrer un extrait de la topologie STP au sein d'un VLAN dans le réseau, examinons là pour le VLAN 10 sur SW1. Chaque Switch possède une carte, de son point de vue, du domaine STP.

```
SW1#sh sp

VLAN0010
  Spanning tree enabled protocol ieee
  Root ID     Priority    32778
              Address     5254.000b.8555
              Cost        4
              Port        2 (GigabitEthernet0/1)
              Hello Time   2 sec  Max Age 20 sec  Forward Delay 15 sec

  Bridge ID   Priority    32778  (priority 32768 sys-id-ext 10)
              Address     5254.000d.2dbe
              Hello Time   2 sec  Max Age 20 sec  Forward Delay 15 sec
              Aging Time  300 sec

Interface           Role Sts Cost      Prio.Nbr Type
------------------- ---- --- --------- -------- --------------------------------
Gi0/0               Desg FWD 4         128.1    P2p
Gi0/1               Root FWD 4         128.2    P2p
Gi0/2               Desg FWD 4         128.3    P2p
```

Figure 3.5 : Domaine PVST+ de SW1 pour le VLAN 10

Nous constatons que ce Switch n'est pas le Root Bridge pour ce VLAN, l'interface root est le port *G0/1* qui pointe directement vers SW3, le *Root Bridge*. Le *Coût Root* vers le *Root Bridge* est également indiqué. Le coût de 4 nous montre que le *Root Bridge* est un voisin direct de SW1. Notons que cette configuration illustre la topologie de base avec les VLANs 10,20,30.

Toutes les autres interfaces du Commutateur sont placées dans l'état Forwarding.

Premier essai de redondance :

Le protocole IEEE 802.1D n'est pas réputé pour son efficacité, essayons de nous rendre compte de la vitesse de re-convergence de la topologie après une modification. Pour cela, reprenons l'exemple de SW1 dont l'interface G0/1 est l'accès direct au *Root Bridge*, comme démontré *fig. 3.5*.

Rappelons que le *Root Bridge* transmet des *BPDUs de configuration* (*Hello BPDUs)* toutes les deux secondes, l'intervalle par défaut. **Ces *BPDUs* sont envoyés sans bits d'indication dans une situation régulière**, puis, les BPDUs sont relayés par les autres Switches *via* leur *designated ports*. Seul le Root Bridge envoie des BPDUs à intervalle régulier en situation normale.

Notons que les commutateurs avec un rôle normal reçoivent les BPDUs du Root Bridge sur leurs ports Root puisque cette interface est censée être l'accès le plus rapide vers ce Commutateur Root pour le domaine de spanning-tree.

Deux types de bits d'indication existent au sein de ce BPDU de configuration pour le protocole 802.1D :

- *Topology Change Acknowledgment (TCA)* : Envoyé après réception d'un message *TCN, tous les switches envoient un BPDU avec ce bit d'indication activé après la réception d'un BPDU de type TCN*

- *Topology Change (TC) :* Bit d'indication placé dans les BPDUs de configuration (ceux envoyés régulièrement par le Root Bridge) pendant une durée égale à la somme du *Max age timer* et du *Forward Delay Timer* (20 + 15 secondes par défaut).
  Sert à indiquer aux Switches qu'il y a eu un changement dans la topologie, de fait, les adresses MAC doivent être rapidement retirées des Tables car ces dernières ne sont plus valides après modification du domaine

Il existe un autre type de BPDU utilisé par ce protocole, le **BPDU TCN** (Topology Change Notification), différent d'un BPDU de configuration classique. Ce message permet d'indiquer au Root Bridge qu'un changement a eu lieu dans la topologie. Ce type de BPDU est envoyé par un Switch qui n'est pas Root Bridge, par le biais de son interface Root lorsqu'un changement est détecté.

La figure 3.7 illustre le parcourt de BPDUs de configuration & BPDUs de type TCN lors d'un changement dans la topologie.[3]

[3] https://networkengineering.stackexchange.com/questions/10015/what-flags-are-used-in-bpdus-in-stp

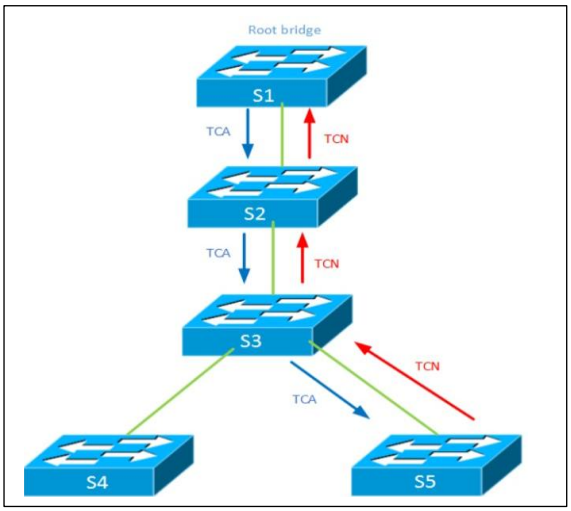


Figure 3.7 : Parcourt des différents BPDUs dans un domaine type suivant un changement

De plus, les interfaces ne peuvent directement passer de l'état Blocking à Forwarding, il y a deux étapes intermédiaires où le port est placé dans un rôle instable, d'abord *Listening* puis *Learning* avant d'être placé dans l'état de Forwarding. L'interface reste dans un état instable pendant une durée de 30 secondes (15 secondes par état), on appelle ce temps le *Forward Delay Timer*.

Simulons désormais une panne sur le segment reliant SW1 et SW3, pour ce faire, l'interface G0/1 de SW1 va être mise administrativement hors-service. Il s'agit du port Root de SW1, ce dernier perdra donc son accès au Root Bridge et va se déclarer lui-même le nouveau Root Bridge avant de recevoir un BPDU avec un *BID* inférieur au sien, pour ensuite ajuster sa configuration et accepter le meilleur Switch en tant que Root Bridge.

```
SW1# configure terminal
SW1(config)# interface g0/1
SW1(config-if)# shut
```

Bridge, se déclarant alors lui-même Root Bridge pour le domaine.

Ensuite, SW1 commence à envoyer des BPDUs de configuration par le biais de ses interfaces, se déclarant ainsi *Root Bridge*. Il est possible de constater cet envoi en examinant le trafic reçu par les voisins de SW1 qui a perdu son chemin vers le Root Bridge.

```
SW4#
*Feb 10 12:38:39.664: STP: VLAN0010 heard root 32778-5254.000d.2dbe on Gi0/
*Feb 10 12:38:41.679: STP: VLAN0010 heard root 32778-5254.000d.2dbe on Gi0/
*Feb 10 12:38:43.710: STP: VLAN0010 heard root 32778-5254.000d.2dbe on Gi0/
```

Figure 3.7.1 : Réception de BPDUs inférieurs émis par SW1 sur SW4

Comme prévu, SW4 reçoit ces BPDUs sur l'interface qui le lie avec SW1, nous reconnaissons en plus, la priorité de SW1 avec son adresse MAC. Il est possible de confirmer la correspondance des adresses en examinant la configuration PVST de SW1 *fig. 3.5* et le *tableau 3.4*.

Tableau 3.4 : Extrait des BPDUs envoyés par SW1 lors de la panne

MAC	Priorité
5254.000D.2DBE	**32778**

SW4 ignore les nouveaux BPDUs car ces derniers sont inférieurs à ceux toujours envoyés par SW3, Root Bridge pour le VLAN 10.

Nous pouvons examiner le trafic entre SW1 et SW4 sur le segment diagonal de la topologie, *fig.1.2*. La commande : *show interface [interface]* nous donne les adresses MAC sources des Trames envoyées.

SW1 G0/2 est identifiée par : 5254.0017.2b2b, identifiant STP global : 5254.000d.2dbe

SW4 G0/3 est identifiée par : 5254.000e.00be, identifiant STP global : 5254.0014.c2af

En s'attachant à un premier BPDU relayé par SW1, *fig. 3.8*, **avant le changement** dans la topologie, nous constatons que le *Bridge Identifier* et le *Root Identifier* sont différents, ce qui est cohérent car SW1 n'est pas le Root Bridge pour le VLAN 10, l'adresse MAC source correspond à l'identification ci-dessus.

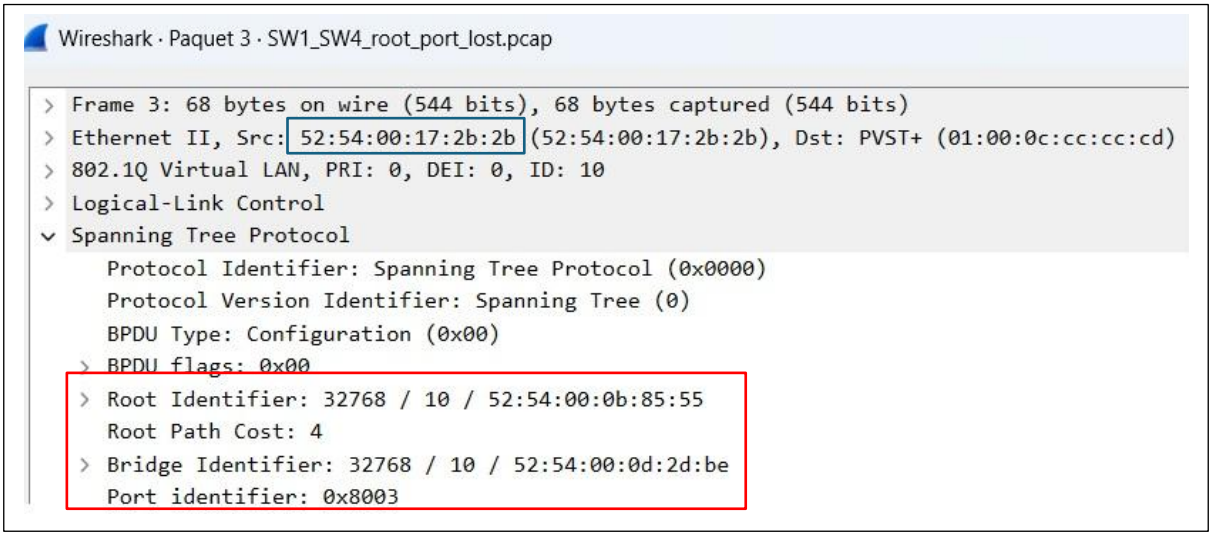


Figure 3.8 : BPDU relayé par SW1 provenant du Root Bridge SW3 avant le changement dans la topologie.

Suivant le changement dans la topologie, les BPDUs de SW1 sont différents, le *Root Identifier* et le *Bridge Identifier* sont identiques et correspondent à l'adresse MAC de SW1. Nous comprenons qu'il se déclare nouveau *Root Bridge* pour ce VLAN, comme confirmé par la réception de ces nouveaux BPDUs sur l'interface de SW4.

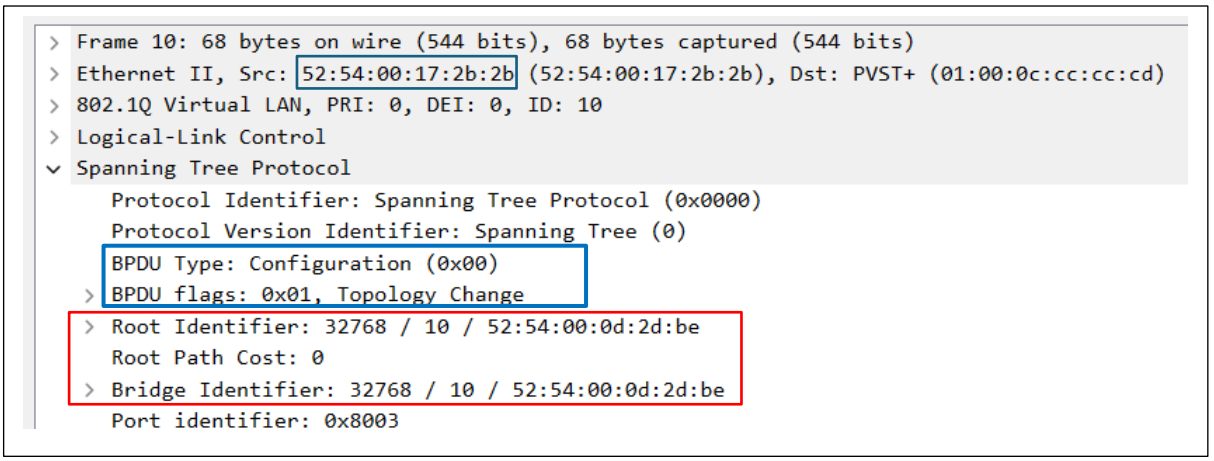


Figure 3.9 : BPDU envoyé par SW1 via G0/2 après la panne, reçu sur l'interface G0/3 de SW4. SW1 se déclare Root Bridge

Le bit d'indication *TC* est également présent dans ce BPDU de configuration, les identifiants du Switch sont équivalent à l'identifiant de SW1 visible *fig.3.5,* ce qui est cohérent avec la situation.

Ensuite, après expiration du délai *Max* Age, SW1 reçoit un BPDU supérieur aux siens (avec une valeur de BID inférieure) et reprend SW3 comme *Root Bridge* pour le VLAN 10, la *figure 3.10* illustre la réception de ce BPDU par SW1.

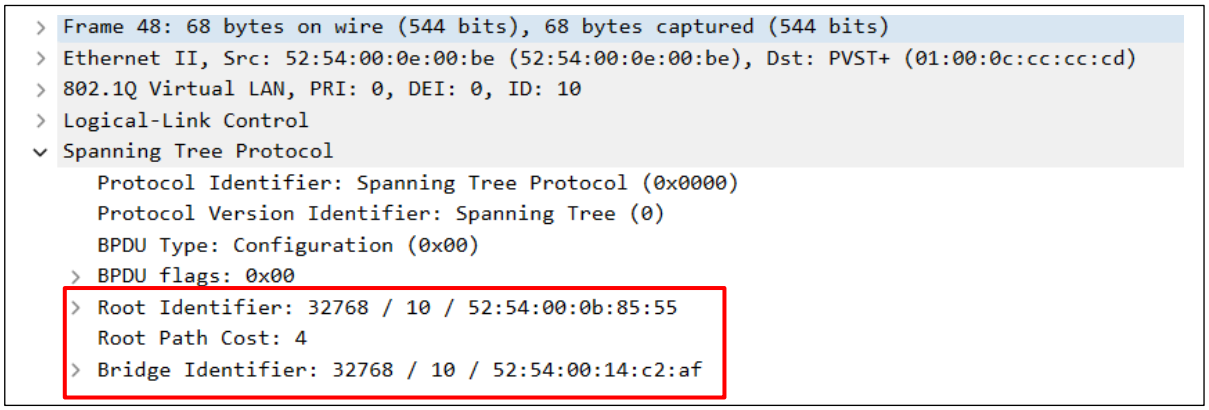


Figure 3.10 : BPDU de configuration de SW3 relayé par SW4 à SW1 sur le segment diagonal après expiration du Max Age timer. Capturé sur l'interface G0/2 de SW1

Nous pouvons ensuite confirmer que SW1 a accepté SW3 comme Root Bridge de nouveau en examinant la configuration STP de SW1 *figure 3.11*. L'identifiant Root 5254.000b.8555 est celui de SW3.

```
SW1(config-if)#do sh sp

VLAN0010
  Spanning tree enabled protocol ieee
  Root ID     Priority    32778
              Address     5254.000b.8555
              Cost        8
              Port        3 (GigabitEthernet0/2)
              Hello Time   2 sec  Max Age 20 sec  Forward Delay 15 sec
```

Figure 3.11 : Configuration STP de SW1 après résolution de la panne

Donc, SW1 reprend son ancien Root Bridge avec un nouveau coût de 8, puisqu'un nouveau chemin a été calculé *via G0/2* comme meilleur chemin vers le Root Bridge.

SW1 envoie ensuite un *TCN via* G0/2 pour indiquer un changement dans la topologie. Ce TCN est retrouvable dans la capture de paquets, *figure 3.12, Frame 49*.

```
47 38.499887    52:54:00:17:2b:2b   PVST+   STP   68 Conf. TC + Root = 32768/10/52:54:00:0d:2d:be  Cost = 0
48 38.507244    52:54:00:0e:00:be   PVST+   STP   68 Conf. Root = 32768/10/52:54:00:0b:85:55  Cost = 4  Port
49 38.514741    52:54:00:17:2b:2b   PVST+   STP   60 Topology Change Notification
50 40.061750    52:54:00:0e:00:be   PVST+   STP   68 Conf. TC + Root = 32768/10/52:54:00:0b:85:55  Cost = 4
```

Figure 3.12 : Capture de Paquets entre SW1 et SW4 sur le segment diagonal

Enfin, SW4 fait basculer son interface dans un état Forwarding.

```
*Feb 10 12:56:07.816: STP: VLAN0010 heard root 32778-5254.000d.2dbe on Gi0/3
*Feb 10 12:56:08.839: STP: VLAN0010 Gi0/3 -> listening
*Feb 10 12:56:09.563: STP: VLAN0010 heard root 32778-5254.000d.2dbe on Gi0/3
*Feb 10 12:56:09.571: STP: VLAN0010 Topology Change rcvd on Gi0/3
*Feb 10 12:56:09.572: STP: VLAN0010 sent Topology Change Notice on Gi0/2
*Feb 10 12:56:23.840: STP: VLAN0010 Gi0/3 -> learning
*Feb 10 12:56:38.839: STP[10]: Generating TC trap for port GigabitEthernet0/3
*Feb 10 12:56:38.840: STP: VLAN0010 sent Topology Change Notice on Gi0/2
*Feb 10 12:56:38.840: STP: VLAN0010 Gi0/3 -> forwarding
```

Figure 3.13 : Logs de SW4 qui bascule G0/3 dans l'état Forwarding après la désactivation de l'interface G0/1 de SW1

Nous constatons le passage par les états intermédiaires avant d'atteindre l'état Forwarding. Le temps de modification de la topologie avant d'être opérationnelle est de 50 secondes, ce qui peut se révéler dommageable pour le bon fonctionnement du réseau. Prenons l'exemple d'un site traitant un taux de trafic important, presque une minute de panne peut impacter des milliers d'utilisateurs.

Solutions pour accélérer la re convergence dans une topologie 802.1D :

Comme démontré, le temps de re convergence du protocole 802.1D est très long, il est nécessaire d'utiliser des solutions qui permettent une re convergence plus rapide de la topologie afin d'éviter ces longues pannes lors d'un changement quelconque.

Plusieurs méthodes existent au sein même du protocole 802.1D afin de limiter les temps de re convergence après un changement, nous pouvons examiner en détails deux de ces solutions.

La première solution se nomme ***UplinkFast***, il s'agit d'une fonctionnalité du protocole 802.1D qui permet de faire passer une interface de l'état Blocking à l'état Forwarding sans délai, sans passer par les états intermédiaires Listening et Learning. Cela permet d'économiser trente secondes de délai avant que la topologie ne soit capable de fonctionner à nouveau.

UplinkFast gère les pannes directes sur des segments et doit être configuré sur des Switches avec des ports dans l'état Blocking, par conséquent des Switches qui ne sont pas Root Bridges.

UplinkFast modifie les priorités des interfaces et la priorité globale du Switch, *voir fig. 3.14*. La priorité du Switch devient 41952 par défaut, priorité à laquelle on ajoute l'identifiant de VLAN. Alors la priorité de SW4 pour le VLAN 20 deviendrait 41952 + 20, soit 41972. De plus, chaque priorité d'interface se voit incrémentée de 3000.

```
VLAN0020
  Spanning tree enabled protocol ieee
  Root ID     Priority    32788
              Address     5254.000a.c095
              Cost        3004
              Port        2 (GigabitEthernet0/1)
              Hello Time   2 sec  Max Age 20 sec  Forward Delay 15 sec

  Bridge ID   Priority    49172  (priority 49152 sys-id-ext 20)
              Address     5254.0014.c2af
              Hello Time   2 sec  Max Age 20 sec  Forward Delay 15 sec
              Aging Time  300 sec
  Uplinkfast enabled

Interface           Role Sts Cost      Prio.Nbr Type
------------------- ---- --- --------- -------- --------------------
Gi0/1               Root FWD 3004      128.2    P2p
Gi0/2               Altn BLK 3004      128.3    P2p
Gi0/3               Desg FWD 3004      128.4    P2p
Gi2/2               Desg FWD 3004      128.11   P2p
```

Figure 3.14 : Changements de priorités suivant UplinkFast sur SW4

Pour démontrer l'efficacité d'UplinkFast, essayons de mettre l'interface Root (G0/1) de SW4 hors-service, simulant une panne, *figure 3.15*.

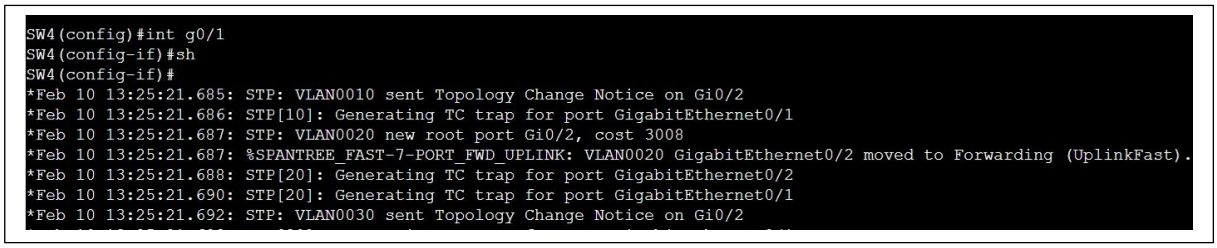


Figure 3.15 : Changement d'état immédiat de G0/2 suivant UplinkFast

Comme démontré par les logs de Spanning-Tree, l'interface G0/2 de SW4 bascule immédiatement dans l'état Forwarding sans passer par les états intermédiaires.

*Feb 10 13:25:21.687: %SPANTREE_FAST-7-PORT_FWD_UPLINK: VLAN0020 GigabitEthernet0/2 moved to Forwarding (UplinkFast).

La transition s'effectue immédiatement, sans aucune perte de Trame circulant *via* ces liens, démontrant l'efficacité et l'importance d'UplinkFast au sein d'une topologie STP.

La deuxième solution est utilisée pour gérer les incidents indirects, c'est-à-dire les pannes qui ne concernent pas directement un Switch, mais plutôt un autre segment de la topologie. Cette solution se nomme ***BackboneFast***, son utilisation permet de passer outre le *Max Age* timer sans avoir besoin d'attendre 20 secondes.

Pour rappel, lorsqu'un incident fait surface sur un segment, le Switch X ayant perdu sa connexion avec le Root Bridge se déclarera lui-même Root Bridge et enverra des BPDUs de configuration aux Switches voisins (Switch Y).

Avant de rectifier la topologie, le *Max Age* timer devra expirer, provoquant vingt secondes de maintenance ajoutées au Forward Delay timer de trente secondes.

BackboneFast permet de passer outre du *Max Age* timer avec un échange de messages : en envoyant une requête *Root Link Query* (*RLQ Request*) au Root Bridge *via* l'interface Root du Switch adjacent. Cette requête permet au Switch Y adjacent qui reçoit les BPDUs erronés de confirmer que le Switch Z est toujours le Root Bridge pour la topologie. En réponse le Root Bridge, Switch Z, enverra une réponse *Root Link Query* (*RLQ Response*) confirmant qu'il est toujours le Root Bridge pour la topologie.

Le schéma ci-dessous, *figure 3.16*, illustre le comportement de la topologie après utilisation de l'outil BackboneFast qui doit être configuré sur tous les Switches du domaine STP.

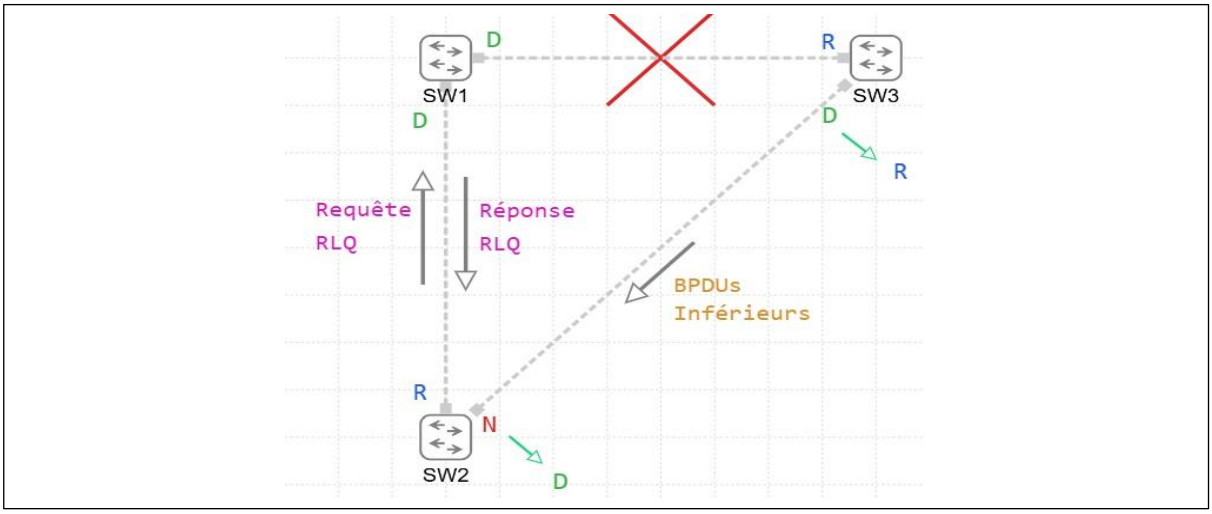


Figure 3.16 : Illustration du fonctionnement de BackboneFast

D'après l'illustration 3.16, SW3 perd sa connexion au Root Bridge (SW1) et se déclare lui-même Root Bridge auprès de SW2. SW2 qui a toujours sa connexion à SW1 interroge ce dernier pour vérifier qu'il est toujours le Root Bridge pour le domaine (RLQ Request), puis SW1 confirme (RLQ Response).

De ce fait, lors d'un incident, des requêtes propres à BackboneFast sont envoyées afin de vérifier l'intégrité du Root Bridge avant de poursuivre la re convergence de la topologie. Nous pouvons examiner ces requêtes et réponses *RLQ* au sein d'une capture de paquets.

La capture de paquets s'effectue sur le segment liant SW3 et SW4, accentué sur l'illustration. L'objectif est de confirmer l'envoi et la réception de messages RLQ. La topologie *figure 1.2* est utilisée, reproduite sur les schémas *figure 3.17* ci-dessous.

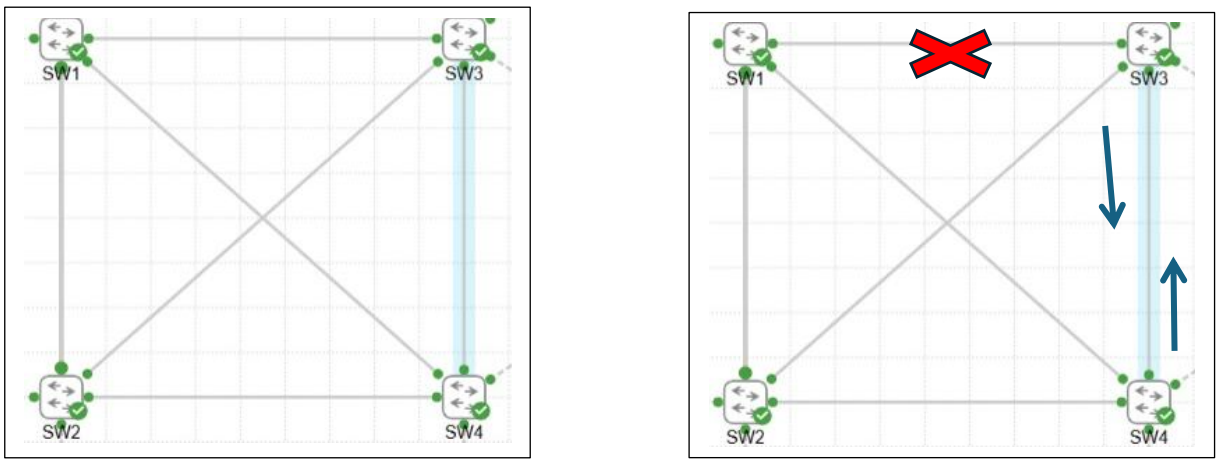


Figure 3.17 : Illustration de l'exemple avec utilisation de BackboneFast

Commençons par simuler la panne sur le segment reliant SW1 et SW3, comme à l'accoutumée :

```
SW1(config)# spanning-tree backbonefast
SW1# configure terminal
SW1(config)# interface G0/1
SW1(config-if)# shut
```

SW1 envoie immédiatement des BPDUs à SW4 sur le segment diagonal, comme visible ci-dessous. Seulement, il n'y a qu'un BPDU envoyé car suivant BackboneFast, le Max Age timer expire directement, l'interface bascule dans les états intermédiaires du Forward Delay timer.

```
SW4(config)#sp backbonefast
*Feb 11 09:52:21.232: STP: VLAN0010 heard root 32778-5254.000d.2dbe on Gi0/3
*Feb 11 09:52:21.238: STP: VLAN0010 Gi0/3 -> listening
*Feb 11 09:52:22.061: STP: VLAN0010 Topology Change rcvd on Gi0/3
*Feb 11 09:52:22.063: STP: VLAN0010 sent Topology Change Notice on Gi0/2
*Feb 11 09:52:36.240: STP: VLAN0010 Gi0/3 -> learning
*Feb 11 09:52:51.244: STP[10]: Generating TC trap for port GigabitEthernet0/3
*Feb 11 09:52:51.248: STP: VLAN0010 sent Topology Change Notice on Gi0/2
*Feb 11 09:52:51.248: STP: VLAN0010 Gi0/3 -> forwarding
```

De fait, l'interface G0/3 de SW4 bascule dans le rôle Listening moins d'une seconde après avoir reçu un BPDU inférieur de SW1. Cela est dû aux messages RLQ échangés entre SW4 et SW3, comme démontré respectivement par les *figures 3.19 et 3.20, frames 14 & 15*.

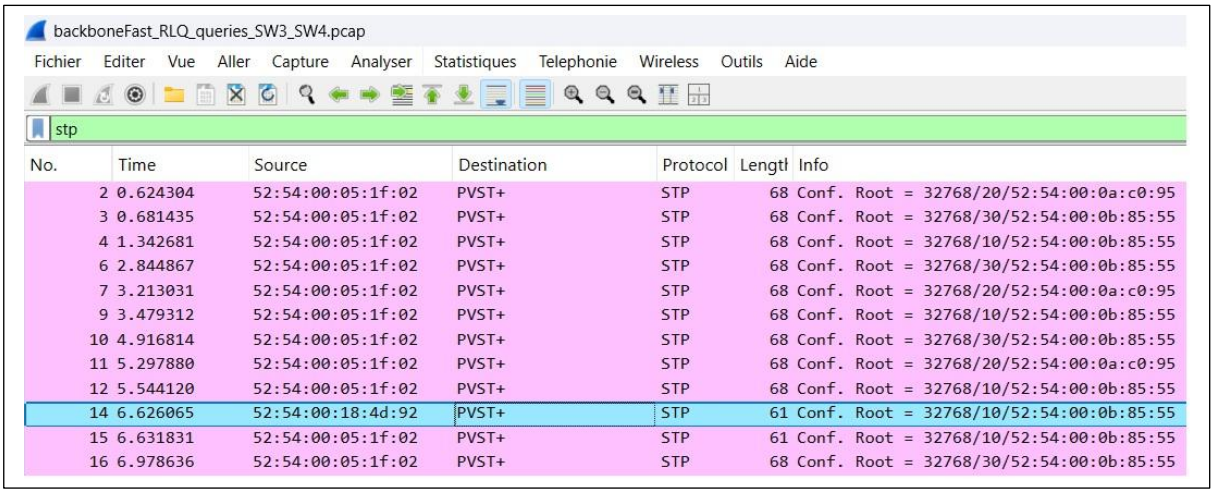


Figure 3.18 : Echange de Trames entre SW3 et SW4

Comme illustré, les échanges sont habituels de la Trame n° 2 à n° 12, il s'agit d'envois de BPDUs de configuration de la part du Root Bridge, SW3. La Trame n°14 (en bleu) est inhabituelle, car envoyée par SW4. Cette Trame survient après la désactivation de l'interface de SW1.

SW4 a reçu d'autres BPDUs (inférieurs) et interroge le Root Bridge, *figure 3.19*, SW3. Examinons cette Trame et la suivante, censée être la réponse de SW3.

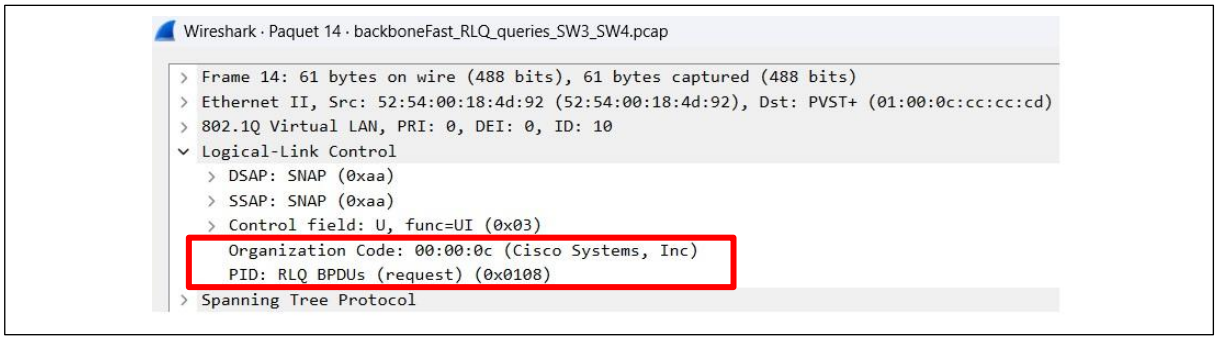


Figure 3.19 : Requête RLQ initiée par SW4 après réception de BPDUs inférieurs émis par SW1

La réponse de SW3 est immédiatement disponible dans la Trame suivante (n°15), *fig.3.20*.

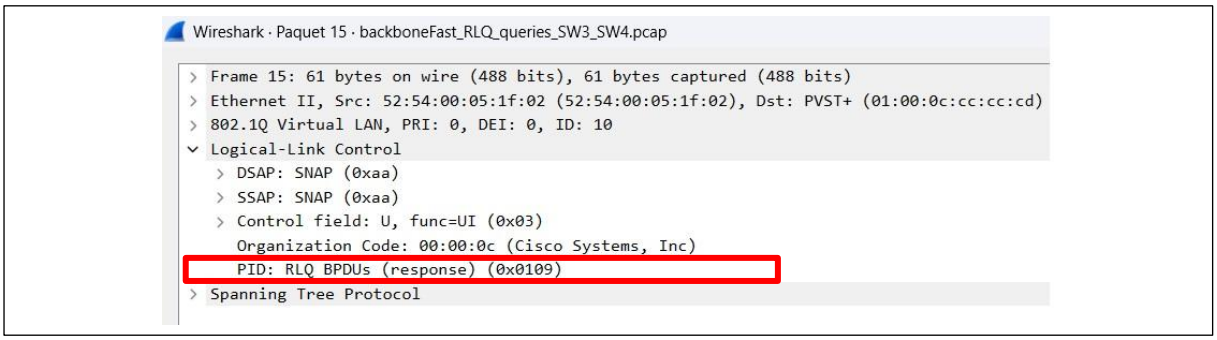


Figure 3.20 : Réponse RLQ de SW3 suivant la requête de SW4

Notons qu'il a fallu déployer des trésors d'ingéniosité afin de trouver ces messages RLQ au sein de la capture. En effet, BackboneFast étant un protocole propriétaire Cisco, ce dernier utilise une encapsulation *SNAP* au lieu du standard *STP Ethernet*. Ces échanges de données permettent de comprendre le fonctionnement de l'outil BackboneFast ainsi que son efficacité. La conclusion de cet outil est le passage d'une maintenance de 50 secondes à 30 secondes en ignorant le Max Age timer de 20 secondes.

D'autres options – qui ne seront pas détaillées ici – existent afin d'améliorer l'efficacité du protocole 802.1D telles que *RootGuard, LoopGuard, BPDUGuard*, etc.

Démonstration d'un Broadcast Storm :

Ainsi, le protocole IEEE 802.1D permet d'éviter des tempêtes Broadcast au sein de LANs rendant son utilisation impérative. Pour démontrer l'effet dévastateur d'un tel phénomène sur un LAN, prenons l'exemple d'un schéma de LAN classique avec des liens redondants entre les Switches, *voir fig.3.21*.

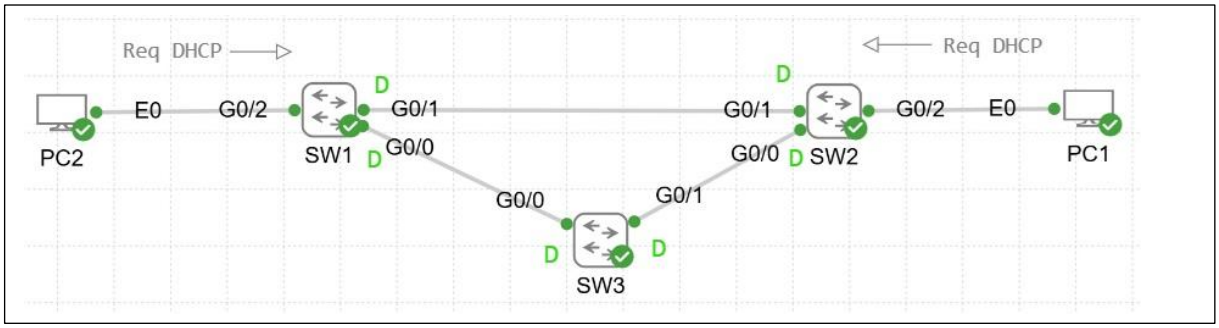


Figure 3.21 : Topologie de démonstration d'une tempête Broadcast comportant des liens redondants, Boucles

Par défaut, STP va désactiver administrativement ces liens comme présenté. Afin de simuler un réseau sans protocole de Layer 2 visant à éliminer ces boucles. L'outil *BPDU Filter* sera utilisé sur les segments entre deux Switches. Cet outil permet à un Switch d'arrêter l'envoi de BPDUs, comme si STP n'était pas utilisé.

L'utilisation de *BPDU Filter* permet de créer une topologie avec des boucles malgré le protocole de Layer 2 visant à les éliminer. Le trafic *Broadcast* sera automatiquement généré par les clients du LAN (PC1 & PC2), envoyant naturellement des requêtes *DHCP* afin d'obtenir une adresse réseau. Il s'agit de trafic Broadcast qui sera relayé et multiplié entre les Switches, provoquant une tempête Broadcast.

Nous pourrons nous rendre compte de l'ampleur de ce phénomène par le biais de capture de paquets et en examinant les statistiques de Trames envoyées par les différentes interfaces des Switches du LAN.

Commençons par configurer *BPDU Filter* sur chaque segment liant deux Switches, cet outil s'utilise par interface ou de manière globale comme tel :

SW# *configure terminal*

SW(config)#*interface [*interface_id*]*

SW(config-if)# *spanning-tree bpdufilter enable*

Une fois ces commandes appliquées sur chaque Switch, plus aucun BPDU ne sera envoyé, tous les ports passeront donc dans l'état Forwarding. Cela provoquera inévitablement des boucles au sein de la topologie entrainant des tempêtes broadcast.

Examinons désormais une capture de paquet entre deux Switches, rappelons que tous les chemins sont actifs, les protocoles censés éliminer les boucles sont désactivés, *fig. 3.23*.



No.	Time	Source	Destination	Protocol	Length	Info
83	0.533588	0.0.0.0	255.255.255.…	DHCP	342	DHCP Discover - Transaction ID 0x9a3db811
86	0.548937	0.0.0.0	255.255.255.…	DHCP	342	DHCP Discover - Transaction ID 0x9a3db811
88	0.556527	0.0.0.0	255.255.255.…	DHCP	342	DHCP Discover - Transaction ID 0x9a3db811
90	0.562945	0.0.0.0	255.255.255.…	DHCP	342	DHCP Discover - Transaction ID 0x9a3db811
92	0.568488	0.0.0.0	255.255.255.…	DHCP	342	DHCP Discover - Transaction ID 0x9a3db811
93	0.574642	0.0.0.0	255.255.255.…	DHCP	342	DHCP Discover - Transaction ID 0x9a3db811
96	0.587036	0.0.0.0	255.255.255.…	DHCP	342	DHCP Discover - Transaction ID 0x9a3db811
98	0.592424	0.0.0.0	255.255.255.…	DHCP	342	DHCP Discover - Transaction ID 0x9a3db811
99	0.596825	0.0.0.0	255.255.255.…	DHCP	342	DHCP Discover - Transaction ID 0x9a3db811
101	0.603543	0.0.0.0	255.255.255.…	DHCP	342	DHCP Discover - Transaction ID 0x9a3db811
104	0.613619	0.0.0.0	255.255.255.…	DHCP	342	DHCP Discover - Transaction ID 0x9a3db811
105	0.617061	0.0.0.0	255.255.255.…	DHCP	342	DHCP Discover - Transaction ID 0x9a3db811
107	0.625388	0.0.0.0	255.255.255.…	DHCP	342	DHCP Discover - Transaction ID 0x9a3db811
110	0.634271	0.0.0.0	255.255.255.…	DHCP	342	DHCP Discover - Transaction ID 0x9a3db811
111	0.637819	0.0.0.0	255.255.255.…	DHCP	342	DHCP Discover - Transaction ID 0x9a3db811
113	0.645778	0.0.0.0	255.255.255.…	DHCP	342	DHCP Discover - Transaction ID 0x9a3db811

Figure 3.23 : Capture du flux de paquet, inondé de trafic Broadcast, de messages DHCP Discover

Ainsi, les liens sont saturés par du trafic broadcast démultiplié à chaque nouvelle occurrence sur un Switch. On constate que plusieurs centaines de paquets sont envoyés et reçus chaque seconde sur les segments qui constituent le LAN.

Conclusion sur le protocole 802.1D :

Ainsi, le protocole Spanning-Tree répond à la première problématique qui consiste à pouvoir utiliser le réseau qui contient de la redondance en la désactivant, empêchant alors les tempêtes Broadcast.

L'utilisation de cette solution apporte de nouvelles problématiques, notamment d'efficacité. Comme démontré, le protocole reconverge de manière significativement lente, environ 50 secondes de maintenance et désactive toute la redondance. C'est-à-dire qu'il ne suffit pas d'ajouter des câbles entre les Switches pour accélérer les transmissions car ces ajouts seront vus comme des boucles, donc automatiquement désactivés par le Spanning-Tree.

Il s'agit désormais de trouver des manières efficientes d'accélérer la re convergence du domaine et de mettre en service la redondance, d'essayer de mobiliser au maximum les boucles du domaine.

Redondance de Layer 2 IEEE 802.1w :

Evolution du protocole IEEE 802.1D :

Le protocole 802.1D a été développé à une époque où les débits supérieurs à 10 Gbps étaient inenvisageables. Depuis, d'autres protocoles, plus efficaces, de gestion de *tempêtes Broadcast* ont été développés. Nous pouvons citer le protocole *Rapid Spanning-Tree* (*RSTP*), *IEEE 802.1w*.

Ce protocole représente une amélioration notable par rapport au protocole STP classique, dans son fonctionnement, particulièrement dans la vitesse de convergence du domaine. En effet, des outils comme *UplinkFast* et *BackboneFast* sont directement implémentés dans RSTP.

Le principe de RSTP est similaire à celui de STP, chaque domaine élit un Root Bridge basé sur le *Bridge ID* de chaque Switch.

Chaque Switch possède un port root pointant directement vers le Root Bridge. Enfin, chaque segment (domaine de collision) est composé d'un port désigné couplé à un port non-désigné.

Les ports non-désignés sont maintenant divisés en deux rôles ; Alternate et Backup. Les apports majeurs de ce nouveau protocole sont les coûts des interfaces, les états et rôles des interfaces, les transitions d'états – RSTP utilise un mécanisme négociation pour les interfaces – des interfaces et la convergence d'un domaine de Layer 2.

En effet, RSTP introduit deux types de coûts pour les ports ; les coûts dits longs et courts.

Tableau 4.1 : Nouveaux coûts propres à RSTP

Vitesse	Coût Court	Coût Long
100 mbps	19	200 000
1 Gbps	4	20 000
10 Gbps	2	2000
100 Gbps	–	200
1 Tbps	–	20

Les coûts courts sont équivalents à ceux utilisé par STP, ces coûts ne sont plus suffisants suivant l'évolution exponentielle de la vitesse des interfaces modernes.

Ainsi, les états des ports sont différents, RSTP rassemble les états Listening et Blocking dans un état appelé Discarding, cet état peut être stable ou instable suivant le rôle de l'interface. Les ports Root et Désignés démarrent dans l'état Discarding, de manière instable, puis évoluent vers l'état Forwarding où ils sont stables. Tandis que les ports avec les rôles Alternate / Backup sont stables dans l'état Discarding, *Voir tableau 4.2*.

L'état Learning est uniquement utilisé lorsque le système de synchronisation RSTP n'est pas valide entre deux Switches. Dans le cas où la synchronisation fonctionne, les ports désignés et Root basculent directement de l'état Discarding vers Forwarding, sinon les ports utilisent l'état Learning.

Tableau 4.2 : Comparaison des états des protocoles 802.1D & 802.1w

Etats Ports STP	Etats Ports RSTP
Blocking	Discarding
Listening	
Learning	Learning
Forwarding	Forwarding

Les rôles d'interfaces sont similaires à ceux utilisés par STP, la seule exception est le rôle Non-désigné qui est divisé en deux sous rôles :

- Alternate : Chemin alternatif prêt à prendre le relai en cas d'incident avec le port Root du Switch

- Backup : Chemin alternatif sur un segment où le Switch a déjà un port désigné, occurrence rare car les Switches sont connectés en P2P. Utilisé avec des *Hubs*

Processus de Synchronisation RSTP :

Comme énoncé, RSTP utilise un mécanisme de synchronisation afin de décider du rôle des ports. Pour rappel, des BPDUs sont utilisés par ces protocoles pour fonctionner.

La structure d'un BPDU est complexe et contient différents champs, comme illustré ci-dessous.

Tableau 4.3 : Représentation d'un BPDU comme utilisé par STP/RSTP[4]

Numéro de champ	Octets	Champs
1	2	Protocol ID
2	1	Version
3	1	Message Type
4	1	Flags
5	8	Root ID
6	4	Coût du chemin
7	8	Identifiant Bridge
8	2	Identifiant d'interface
9	2	Age du message
10	2	Max Age
11	2	Hello Timer
12	2	Forward Delay

Le champ pertinent pour le processus de synchronisation RSTP est *Flags*, ce champ est défini par un Octet où chaque Bit d'indication peut être utilisé.

Dans le cas de STP, seuls les deux premiers bits de l'Octet réservé pour le champ *Flags* sont utilisés, il s'agit du TCA et du TC, expliqués au sein de la section consacrée à STP.

RSTP se sert des huit bits de ce champ pour son processus de synchronisation associé à la négociation des rôles. Comparons les bits du champ *Flags* utilisés par STP et RSTP pour leur fonctionnement.

[4] Voir le détail de la structure et des champs d'un BPDU de configuration utilisé par les normes 802.1D & 802.1w : https://www.networkplayroom.com/2017/09/bpdu-format-and-stp-timers.html

Tableau 4.4 : Composition du Champ Flags d'un BPDU

Numéro de Bits	STP	RSTP
Bit 1	Topology Change (TC)	Topology Change (TC) : Utilisé pour indiquer un changement dans la topologie
Bit 2	TCA	TCA : Réponse à un BPDU de type TCN
Bit 3	–	Proposal : Utilisé dans le processus de synchronisation pour débuter un échange
Bit 4	–	Agreement : Confirme l'accord de synchronisation
Bit 5	–	Learning : Indique un port dans l'état Learning
Bit 6	–	Forwarding : Indique un port dans l'état Forwarding
Bit 7	–	Rôle : Indique si l'émetteur
Bit 8	–	Root : utilisé si l'émetteur est Root Bridge

Commençons par un cas simple, nous avons un segment qui relie deux Switches utilisant RSTP. Chaque Switch commence en plaçant son interface dans le rôle Designated avec l'état Discarding. Ensuite, chaque envoie des BPDUs, le bit numéro 7 (Rôle) est utilisé, il s'agit d'une proposition. SW1 ignore les BPDUs inférieurs aux siens de SW2.

Puis, le Switch 1 avec le BPDU supérieur est élu Root Bridge, le Switch 2 renvoie plusieurs BPDUs avec le bit 4 (Agreement) fixé, confirmant que la proposition de SW1 est acceptée.

Ensuite le port de SW2 bascule dans le rôle root et dans l'état Forwarding, durant ce processus, tous les ports de SW2 connectés à d'autres Switches / Routeurs – *Non-Edge ports* – sont placés dans l'état Discarding. Le port de SW1 reste dans le rôle designated mais bascule dans l'état Forwarding. Le processus d'échange est observable *fig. 4.1, 4.2*.

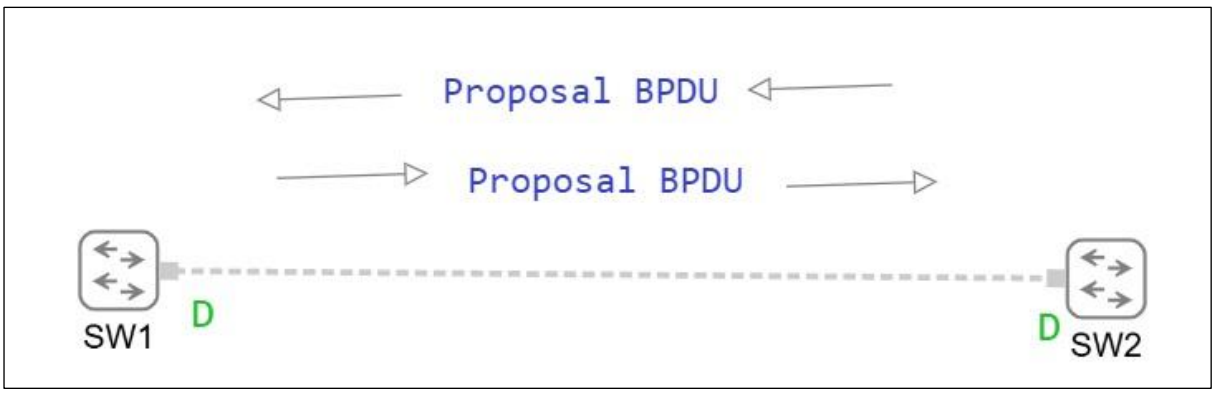


Figure 4.1 : Première Etape de Synchronisation RSTP



Figure 4.2 : Deuxième Etape de Synchronisation RSTP Accord

Le processus d'élection de Root Bridge est sensiblement le même que pour STP, le *BID* le plus faible au sens mathématique l'emporte. Ici, d'autres bits du champ sont utilisés comme *Proposal* (3) et *Agreement (4)*.

Examen des Logs lors du processus de Synchronisation RSTP :

Pour vérifier le fonctionnement de cette synchronisation, nous allons reprendre le scénario précédent sur un unique segment composé de deux Switches, par souci de simplicité. Notons qu'un contexte réaliste ne correspond pas à une synchronisation sur un segment unique.

En amont de l'utilisation de RSTP, la supériorité des BPDUs de SW1 est assurée, sa priorité étant manuellement abaissée à 4096, contre 32768 par défaut. RSTP n'est pas utilisé par défaut, il faut l'activer par Switch comme indiqué :

```
SW# configure terminal
SW(config)# spanning-tree mode rapid-pvst
```

D'une manière similaire à STP, rapid-pvst est une version propriétaire Cisco de la norme *IEEE 802.1w*, cela fonctionne de la même manière que RSTP, or une instance RSTP est utilisée par VLAN plutôt qu'un domaine RSTP global.

Après avoir activé le protocole rapid-pvst, les négociations de rôles apparaissent dans les logs STP de SW1 et SW2, respectivement sur les *fig.4.3* et *4.4*.

```
*Feb 13 10:46:54.095: setting bridge id (which=3) prio 4097 prio cfg 4096 sysid 1 (on) id 1001.5254.0015.dc8f
*Feb 13 10:46:54.095: RSTP(1): initializing port Gi0/1
*Feb 13 10:46:54.096: RSTP(1): Gi0/1 is now designated
*Feb 13 10:46:54.109: RSTP(1): transmitting a proposal on Gi0/1
*Feb 13 10:46:55.097: RSTP(1): transmitting a proposal on Gi0/1
*Feb 13 10:46:55.990: %LINK-3-UPDOWN: Interface GigabitEthernet0/1, changed state to up
*Feb 13 10:46:56.132: RSTP(1): received an agreement on Gi0/1
*Feb 13 10:46:56.142: STP[1]: Generating TC trap for port GigabitEthernet0/1
```

Figure 4.3 : Logs de SW1 lors d'une synchronisation RSTP sur un segment unique

```
*Feb 13 10:46:51.478: setting bridge id (which=3) prio 32769 prio cfg 32768 sysid 1 (on) id 8001.5254.0004.71db

*Feb 13 10:46:51.479: RSTP(1): initializing port Gi0/0

*Feb 13 10:46:51.479: RSTP(1): Gi0/0 is now designated

*Feb 13 10:46:51.510: RSTP(1): transmitting a proposal on Gi0/0

*Feb 13 10:46:52.480: RSTP(1): transmitting a proposal on Gi0/0

*Feb 13 10:46:53.275: %LINK-3-UPDOWN: Interface GigabitEthernet0/0, changed state to up

*Feb 13 10:46:53.298: RSTP(1): updt roles, received superior bpdu on Gi0/0

*Feb 13 10:46:53.299: RSTP(1): Gi0/0 is now root port

*Feb 13 10:46:53.299: RSTP(1): synced Gi0/0

*Feb 13 10:46:53.303: STP[1]: Generating TC trap for port GigabitEthernet0/0

*Feb 13 10:46:53.312: RSTP(1): transmitting an agreement on Gi0/0 as a response to a proposal
```

Figure 4.4 : Logs de SW2, acceptation d'une proposition supérieure (SW1) lors d'une synchronisation RSTP

Ainsi, la synchronisation théorique se vérifie, chaque interface commence dans le rôle Designated, puis transmet des propositions. Le Switch qui reçoit une proposition supérieure – SW2 en l'occurrence – accepte l'autre Switch en tant que *Root Bridge* et renvoie un BPDU avec le bit *Agreement* activé. L'interface de SW2 bascule directement dans le rôle Root tandis que l'interface du Root Bridge reste dans le rôle Designated et bascule dans l'état Forwarding.

D'après les logs, ce processus de communication est extrêmement rapide, le domaine converge en quelques secondes. Cela représente une amélioration considérable en comparaison avec le protocole STP classique où les temps de convergence étaient de 30 ou 50 secondes dans certains cas.

Examen des Trames lors du processus de Synchronisation RSTP :

Les logs pour la convergence précédente ont confirmé le bon fonctionnement du protocole, attachons-nous désormais à l'échange de Trames lors de ce processus afin de visualiser les variations de BPDUs entre STP et RSTP. La capture de paquets est initialisée sur le segment liant SW1 et SW2, les échanges sont représentés *fig. 4.5*, SW1 en rose et SW2 en bleu.

No.	Time	Source	Destination	Protocol	Lengtl	Info
1	0.000000	52:54:00:0a:07:93	Spanning-tre…	STP	60	RST. Root = 4096/1/52:54:00:15:dc:8f Cost = 0 Port = 0x8002
3	1.085169	52:54:00:0a:07:93	Spanning-tre…	STP	60	RST. Root = 4096/1/52:54:00:15:dc:8f Cost = 0 Port = 0x8002
4	1.709276	52:54:00:04:71:db	Spanning-tre…	STP	60	RST. Root = 32768/1/52:54:00:04:71:db Cost = 0 Port = 0x8001
7	3.095689	52:54:00:0a:07:93	Spanning-tre…	STP	60	RST. Root = 4096/1/52:54:00:15:dc:8f Cost = 0 Port = 0x8002
8	3.865015	52:54:00:04:71:db	Spanning-tre…	STP	60	RST. TC + Root = 4096/1/52:54:00:15:dc:8f Cost = 0 Port = 0x8002
9	3.874983	52:54:00:0a:07:93	Spanning-tre…	STP	60	RST. TC + Root = 4096/1/52:54:00:15:dc:8f Cost = 0 Port = 0x8002
11	4.759310	52:54:00:04:71:db	Spanning-tre…	STP	60	RST. TC + Root = 4096/1/52:54:00:15:dc:8f Cost = 4 Port = 0x8001
13	5.111571	52:54:00:0a:07:93	Spanning-tre…	STP	60	RST. TC + Root = 4096/1/52:54:00:15:dc:8f Cost = 0 Port = 0x8002
15	6.788617	52:54:00:04:71:db	Spanning-tre…	STP	60	RST. TC + Root = 4096/1/52:54:00:15:dc:8f Cost = 4 Port = 0x8001
17	7.137637	52:54:00:0a:07:93	Spanning-tre…	STP	60	RST. TC + Root = 4096/1/52:54:00:15:dc:8f Cost = 0 Port = 0x8002
19	9.154207	52:54:00:0a:07:93	Spanning-tre…	STP	60	RST. Root = 4096/1/52:54:00:15:dc:8f Cost = 0 Port = 0x8002
20	11.168814	52:54:00:0a:07:93	Spanning-tre…	STP	60	RST. Root = 4096/1/52:54:00:15:dc:8f Cost = 0 Port = 0x8002
22	13.190096	52:54:00:0a:07:93	Spanning-tre…	STP	60	RST. Root = 4096/1/52:54:00:15:dc:8f Cost = 0 Port = 0x8002
25	15.200158	52:54:00:0a:07:93	Spanning-tre…	STP	60	RST. Root = 4096/1/52:54:00:15:dc:8f Cost = 0 Port = 0x8002
28	17.226243	52:54:00:0a:07:93	Spanning-tre…	STP	60	RST. Root = 4096/1/52:54:00:15:dc:8f Cost = 0 Port = 0x8002
31	19.238568	52:54:00:0a:07:93	Spanning-tre…	STP	60	RST. Root = 4096/1/52:54:00:15:dc:8f Cost = 0 Port = 0x8002

Figure 4.5 : Echange de Trames entre SW1 & SW2, synchronisation RSTP

Comme attendu, après initialisation des interfaces, chaque Switch se déclare *Root Bridge*, ce comportement se confirme avec le champ ; *Root = [priorité] / [adresse MAC]*.

Examinons ces première trames en détails, respectivement Trames 3 & 4.

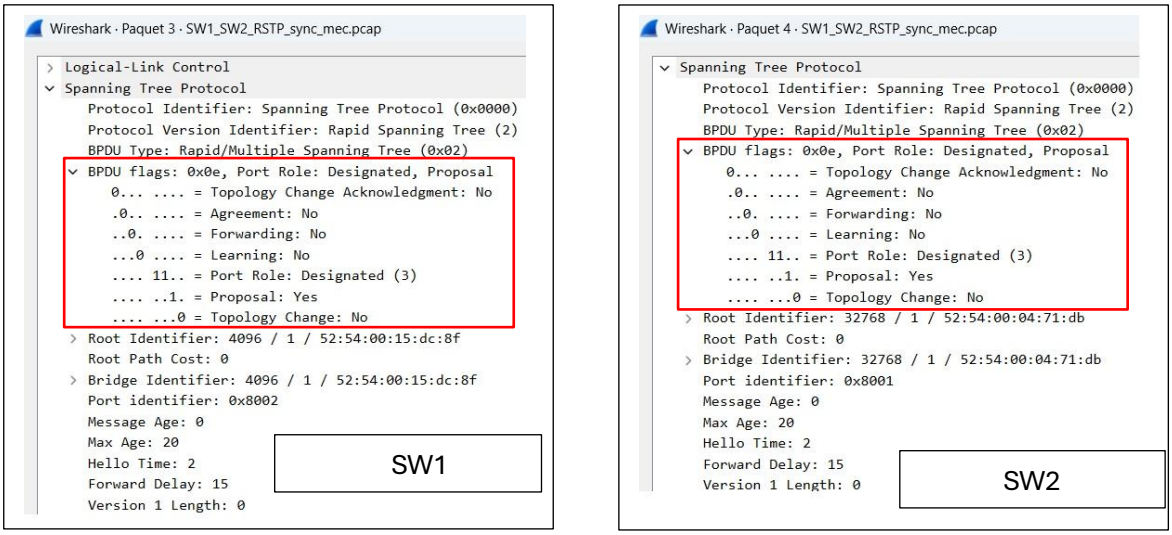


Figure 4.6 : Premières propositions de SW1 & SW2, Trames 3 & 4 de la capture

Les deux ports ont le rôle Designated et le bit *Agreement* est fixé dans les deux Trames, *voir BPDU Flags*. Les priorités et les identifiants Bridge sont illustrés.

Examinons la réaction de SW2 après réception d'un BPDU supérieur de SW1.

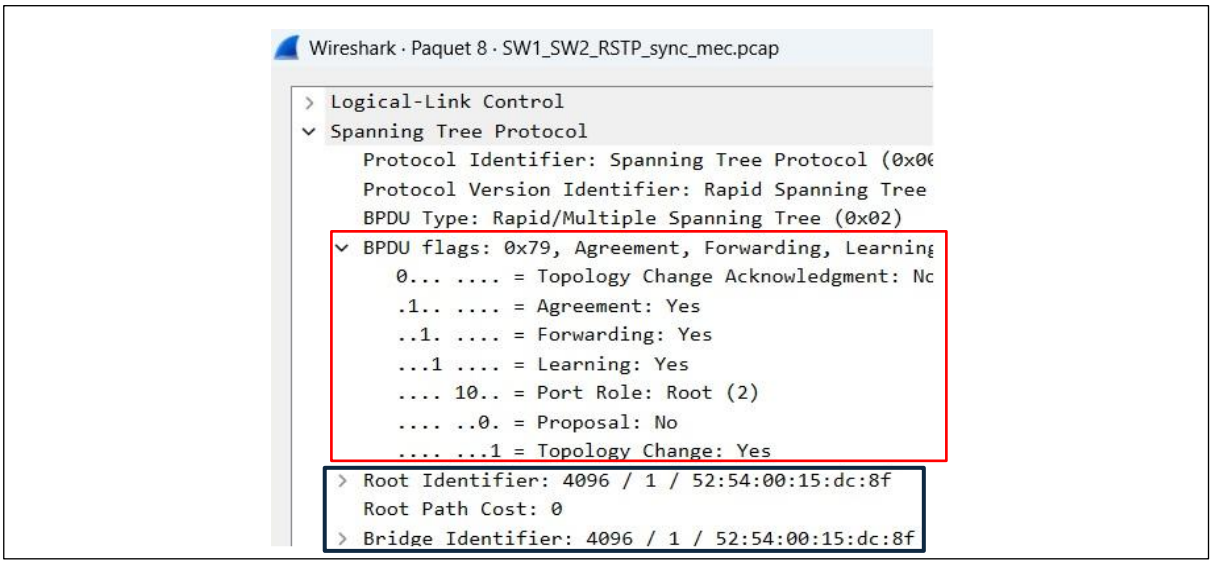


Figure 4.7 : Réponse de SW2 aux BPDUs supérieurs de SW1, Trame 8

Plusieurs nouveaux bits d'indication du champ Flags sont fixés, notamment le bit *Agreement* qui indique que SW2 a accepté la proposition de SW1. Le rôle de l'interface de SW2 a également changé pour basculer dans le rôle Root, ce qui est cohérent avec le bit Learning également fixé qui indique que l'interface est capable d'apprendre les adresses MAC. Enfin, le bit Forwarding est fixé comme le bit TC qui indique un changement dans la topologie.

On remarque une incohérence avec le *Root Identifier* et le *Bridge Identifier* qui se verra corrigée après la réception d'un nouveau message de SW1 avec le bit TC fixé. Cette incohérence entraîne une valeur du *Root Cost* de 0 erronée car SW2 n'est pas Root Bridge.

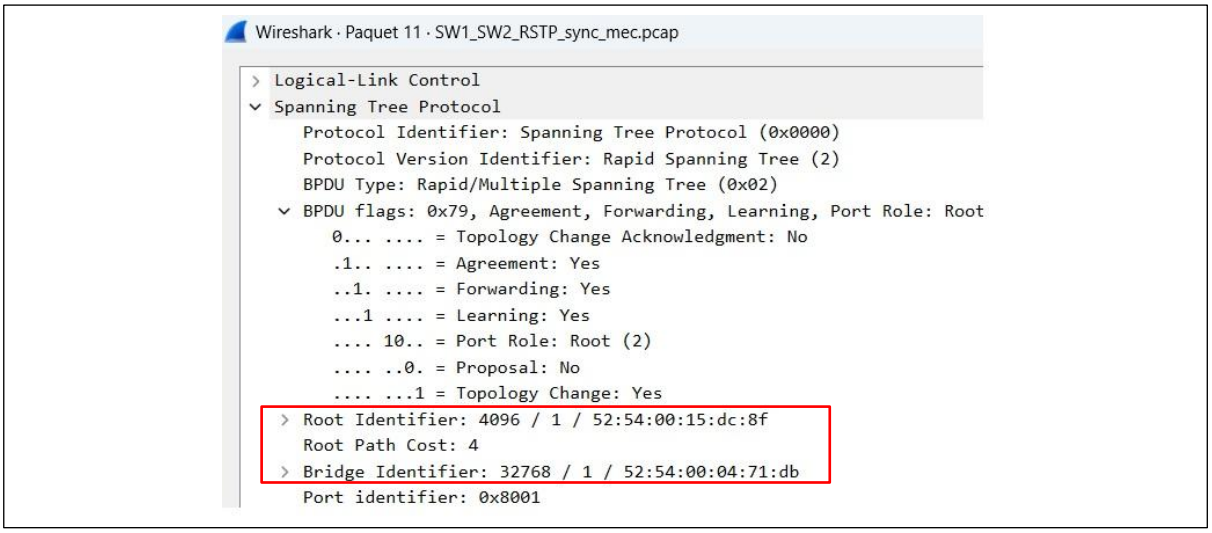


Figure 4.8 : Correction des incohérences d'identifiants Roots et Bridge sur SW2, Trame 11

La trame 11, *fig. 4.8*, indique que les erreurs d'identifiants Root et Bridge ont été corrigées automatiquement suivant la réception de nouveaux BPDUs envoyés par SW1. De fait, le Bridge ID est de nouveau différent du *Root ID* et le coût Root est de 4, ce qui est logique suivant la disposition des Switches dans la topologie.

Gestion d'incident d'un domaine par RSTP :

Pour compléter l'étude de ce protocole au sein d'un LAN, regardons la réponse qu'il apporte aux éventuels incidents. L'avantage majeur de RSTP sur STP est sa vitesse, le temps de convergence d'un domaine devrait être extrêmement rapide afin de limiter le temps de maintenance.

Afin de simuler un incident réaliste, la topologie de base du projet, *fig.1.2, page* 2, sera utilisée avec les quatre Switches mettant en scène le protocole Rapid-pvst. La perte de lien touche le segment liant SW1 à SW3 – le Root Bridge pour le VLAN 10 – où tous les rôles et états des ports ont déjà été négociés. La situation met en évidence une perte de connexion du port Root de SW1, donc de son accès au Root Bridge.

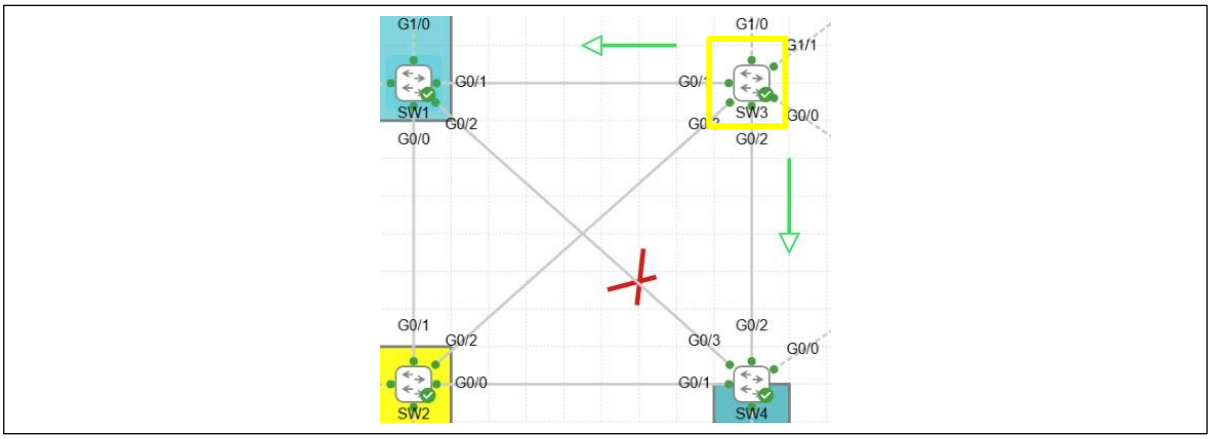


Figure 4.9 : Schématisation de la topologie avant l'incident provoquant la perte du segment liant SW1 à SW3

Pour le moment, l'interface G0/3 de SW4 est dans le rôle *Alternate* associé à l'état Discarding, il s'agit d'un chemin alternatif en cas de perte de connexion sur un autre segment. Supposons que l'interface G0/1 de SW1 soit mise hors service, le Switch perd alors sa liaison avec le *Root Bridge* et s'auto-proclame Root Bridge, envoyant des BPDUs inférieurs à ses voisins. SW4 réagit instantanément en envoyant une proposition à SW1 – Proposal BPDU – qui l'accepte. La topologie a convergé instantanément, *Voir fig. 4.10*.

**Feb 14 12:14:39.058: RSTP(10): updt roles, root port Gi0/1 going down*

**Feb 14 12:14:39.059: RSTP(10): we become the root bridge*

**Feb 14 12:14:39.060: RSTP(10): Gi0/1 not in sync*

**Feb 14 12:14:39.097: RSTP(10): updt roles, received superior bpdu on Gi0/2*

**Feb 14 12:14:39.098: RSTP(10): Gi0/2 is now root port*

**Feb 14 12:14:39.098: RSTP(10): syncing port Gi0/0*

**Feb 14 12:14:39.100: RSTP(10): synced Gi0/2*

**Feb 14 12:14:39.101: STP[10]: Generating TC trap for port GigabitEthernet0/2*

**Feb 14 12:14:39.112: RSTP(10): transmitting an agreement on Gi0/2 as a response to a proposal*

Figure 4.10 : Logs RSTP de SW1 après désactivation de son port root

Les Trames peuvent également être examinées, il s'agit du même processus de synchronisation avec une proposition puis une validation de la proposition.

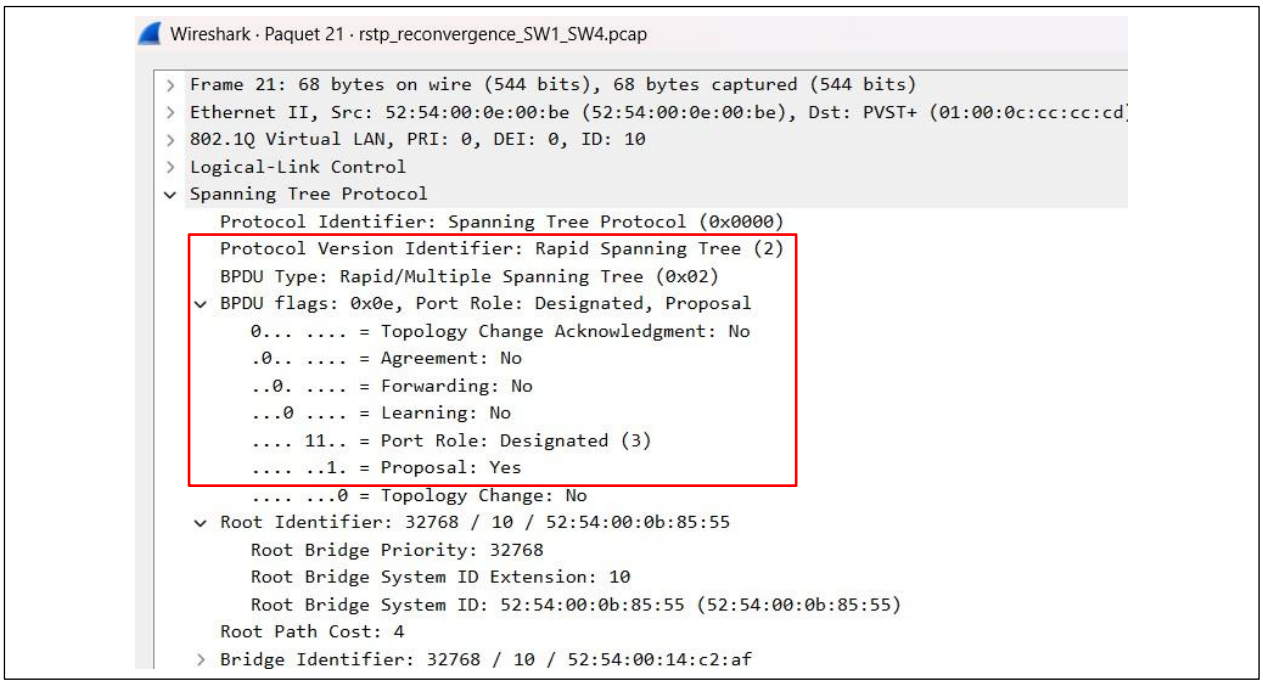


Figure 4.11 : Proposition de SW4 à la suite de réception des BPDUs de SW1, capturée sur le segment diagonal liant SW1 à SW4

Résumé du Rapid Spanning-Tree :

En synthèse, RSTP représente une innovation majeure par rapport à son prédécesseur STP, notamment pour les processus de synchronisation et de convergence. Ce protocole réduit significativement les temps de maintenance et est également compatible avec STP classique, lorsque le processus de synchronisation échoue. De fait, un Switch utilisant une instance RSTP par défaut peut s'adapter à une topologie sollicitant le protocole STP, néanmoins, il sera impossible de profiter de la vitesse et du mécanisme de synchronisation propre à RSTP.

Ainsi, le fonctionnement de RSTP est, *in fine*, sensiblement équivalent à celui du protocole *IEEE 802.1D* à quelques exceptions près. La finalité est la même ; empêcher la formation de *boucles* au sein de la topologie et accélérer les temps de convergence afin de limiter les délais de maintenance.

Or, Deux nouvelles problématiques émergent ; utiliser une instance RSTP / STP par VLAN représente un coût significatif : des *BPDUs* sont envoyés par VLAN toutes les deux secondes (*Hello Timer*). Ces envois répétés mobilisent la capacité d'envoi des Switches de la topologie et peuvent entraver la circulation du trafic. Il semblerait judicieux de pouvoir associer plusieurs VLANs à une seule instance STP dans le but de limiter le trafic inutile.

De plus, la redondance n'est toujours pas correctement gérée, il est impossible d'accélérer la vitesse de transmission et d'augmenter la capacité d'accueil de trafic sur les segments de distributions (liens entre les Switches). Il est essentiel de trouver un moyen de mobiliser les segments inactifs afin d'augmenter le flux de trafic.

Multiple Spanning-Tree :

Pertinence du protocole IEEE 802.1s :

Comme copieusement expliqué, les protocoles IEEE 802.1D & 8021.w utilisent – par VLAN – une instance STP. Les mécanismes de convergence diffèrent, or, la logique d'envois de BPDUs suivant le *Hello Timer* choisi, demeure inchangée. De ce fait, les réseaux comportant un nombre important de VLANs ne sont pas adaptés pour l'utilisation de ces protocoles. Il ne s'agit pas d'incapacité technique, mais plutôt d'un manque de cohérence. En effet, utiliser une instance STP par VLAN est parfaitement raisonnable au sein d'une topologie qui héberge une poignée de VLANs.

Néanmoins, la plupart des réseaux actuels – excluant ceux appartenant à des particuliers – mobilisent des batteries de VLANs pour assurer leur fonctionnement. Il parait alors peu efficient d'utiliser une instance STP par VLAN dont l'envoi régulier de BPDUs saperait une part non-négligeable des ressources des Switches du réseau.

Pour illustrer ce problème, prenons l'exemple d'une topologie composée de trois Switches où 100 VLANs seraient utilisés. Rapid PVST permet d'équilibrer l'envoi de trafic au sein d'un VLAN. Ici, SW1 serait Root Bridge pour les VLANs 1 à 50 et SW2 pour les VLANs 51 à 100. Nous pouvons constater l'envoi de BPDUs pour chaque Root Bridge, *fig. 5.1*.

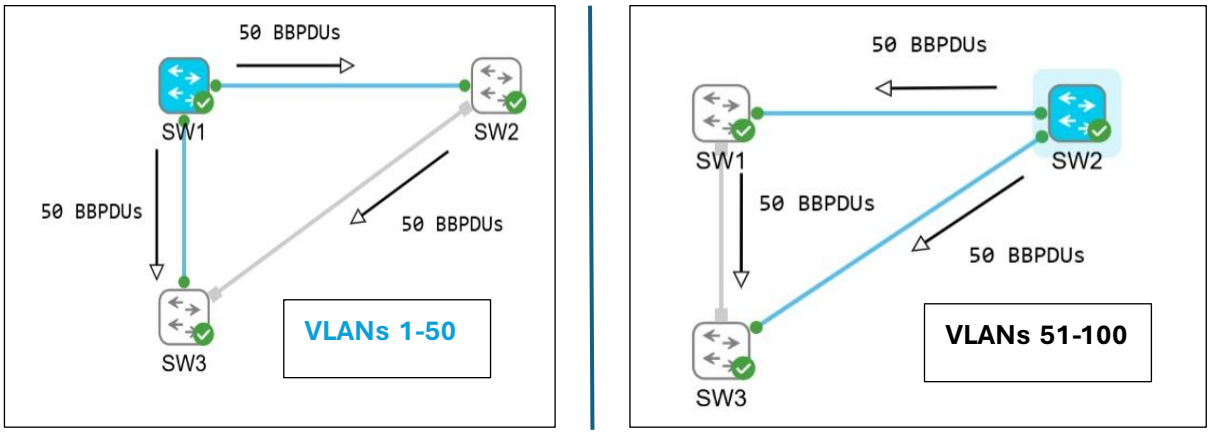


Figure 5.1 : Illustration des BPDUs envoyés avec une instance STP par VLAN

On constate qu'un nombre important de BPDUs est envoyé régulièrement dans la topologie, chaque BPDU doit être traité individuellement par chaque Switch, représentant un coût significatif de ressources. Cela ne semble pas nécessaire car nous constatons qu'il n'y a que deux instances STP différentes au sein du réseau, il parait alors inutile d'avoir 100 instances qui envoient chacune des BPDUs à répétition.

Fonctionnement de MSTP :

MSTP est un protocole utilisé pour gérer les boucles de Layer 2, il s'agit, en quelque sorte, de l'évolution du protocole RSTP, IEEE 802.1w. MSTP utilise les mêmes principes de fonctionnement que RSTP afin d'assurer une rapidité de convergence. L'intérêt majeur de cette mutation est la capacité d'associer une seule instance STP à plusieurs VLANs, réduisant drastiquement le nombre de BPDUs envoyés. Cela se nomme *Instance MST* (*MSTI*), ces *MSTI* permettent de regrouper plusieurs VLANs dans une même configuration.

Pour reprendre l'exemple illustré *fig. 5.1* et atteindre le même équilibre partagé entre deux Switches, MSTP permet de créer uniquement deux instances STP (*MSTIs*), donc de réduire le nombre de BPDUs à chaque Hello Timer, *voir fig. 5.2*.

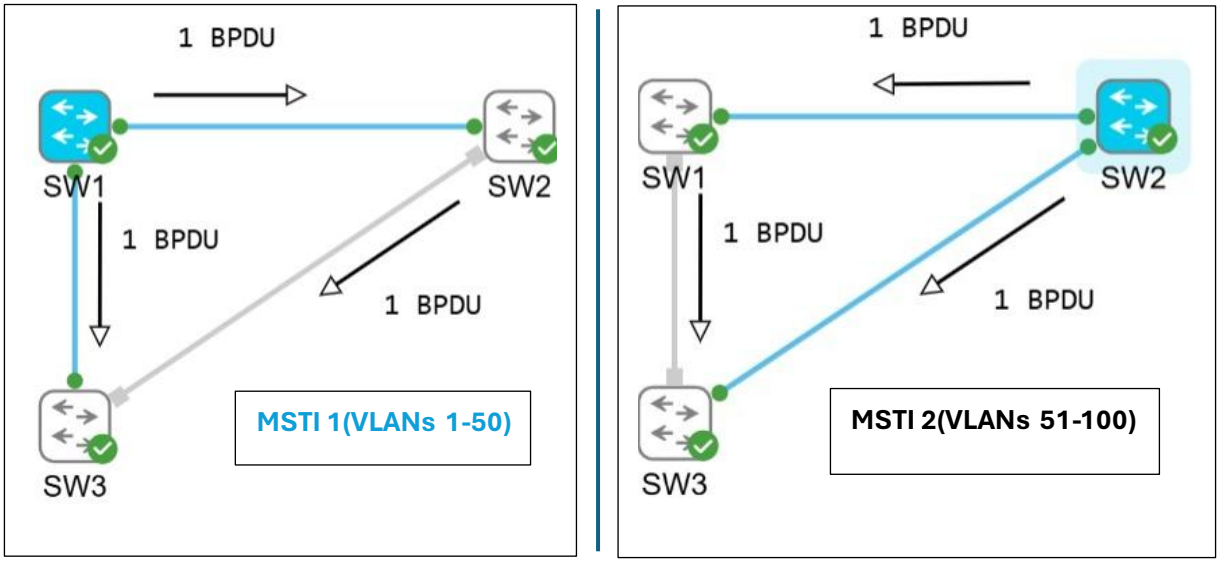


Figure 5.2 : Modification du nombre de BPDUs envoyés suivant les deux instances MSTP

Par défaut, MSTP utilise une instance 0, *MSTI 0* où tous les VLANs sont assignés par défaut. Il n'est pas recommandé d'utiliser cette instance pour des VLANs actifs dans la topologie. De plus, cette instance ne peut être désactivée.

Premier essai de fonctionnement :

Prenons l'exemple d'une topologie simple de trois Switches avec des liens redondants, le protocole MSTP sera mis en place sur tous les Switches, deux instances seront utilisées : *MSTI1*& *MSTI2*, *le tableau 5.1* illustre la répartition des VLANs dans ces instances. On cherche ici à illustrer l'intérêt principal de MSTP ; l'utilisation d'une instance spanning-tree pour plusieurs VLANs. Pour dépeindre au mieux son efficacité, quatre VLANs seront utilisés et des liens de type Trunk seront mis en place entre les Switches.

Tableau 5.1 : VLANs associés aux instances MSTP

Instance 1	Instance 2
VLAN10	**VLAN30**
VLAN20	**VLAN40**

La configuration s'effectue par Switch comme tel :

SW1# *configure terminal*

SW1(config)# *spanning-tree mode mst*

SW1(config)# *spanning-tree mst instance* [identifiant_instance] *priority* [priorité Bridge]

SW1(config)# *spanning-tree configuration*

SW1(config-mst)# *instance* [identifiant_instance] *vlan* [vlan_id]

Pour la configuration, un Switch sera Root pour une instance, le deuxième Switch sera Root pour la seconde instance. Cela permet d'équilibrer les envois de BPDUs.

Chaque Switch Root primaire se verra assigné une priorité de 4096 contrairement à 8192 pour le Switch Root secondaire de l'instance. Cela permet de s'assurer de l'issue des élections de Root Bridge par instance, permettant d'atteindre l'équilibrage de charges voulu ; nous souhaitons que plusieurs chemins différents soient utilisés dans la répartition du trafic de Layer 2, cela permet d'atteindre un équilibre et d'éviter une surcharge des segments du réseau.

Nous obtenons ainsi une configuration MSTP commune et synchronisée entre les Switches de la topologie suivant les trois instances imbriquées des VLANs de la topologie, *voir fig.5.3*. De ce fait, deux *BPDUs* de configuration seront envoyés périodiquement – suivant le *Hello Timer* – dans la région MSTP au lieu de quatre ; soit un BPDU par VLAN. Cette réduction d'envoi de BPDUs est plus marquée lorsque le réseau compte un nombre important de VLANs.

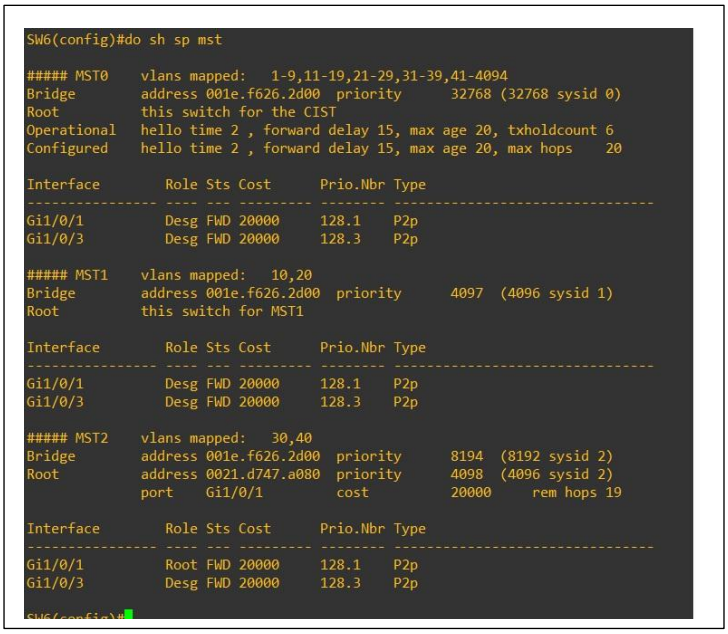


Figure 5.3 : Configuration MSTP d'un Switch de la topologie d'illustration

Cet extrait démontre l'état et le rôle des ports du Switch par instance MST, les priorités des ports sont également différentes des autres protocoles spanning-tree étudiés préalablement. Nous retrouvons également la liste des VLANs associés aux instances. La priorité affichée est différente de celle spécifiée dans les commandes de configuration.

Ce phénomène s'explique car la priorité de chaque bridge est définie par MSTP comme tel : *Priorité_Finale = Priorité_Initiale + numéro d'instance*.

Ainsi, MSTP est bien plus efficient que ses prédécesseurs, en effet, le protocole *802.1s* permet de réduire drastiquement les calculs nécessaires à la maintenance, optimisant alors les ressources du réseau.

Equilibrage des charges :

Dans l'idéal, on aimerait utiliser tous les liens du réseau, mais ce n'est pas possible car cela entraîne des liens redondants qui sont incompatibles avec le trafic Broadcast. MSTP propose une solution à cette problématique et permet d'atteindre une symétrie dans la commutation de trames. Tout cela grâce à un phénomène appelé équilibrage qui permet d'utiliser tous les liens du domaine sans avoir de boucle. Pour cela, on manipule manuellement les élections des Switches Racines afin de forcer certains VLAN à emprunter un passage et les autres VLANs à emprunter le chemin alternatif. Ainsi, tous les liens sont actifs mais il n'y a pas de boucle du point de vu de chaque VLAN ou instance MSTP. De plus, si un chemin vient à lâcher, les VLANs peuvent tous emprunter le même segment dans un état dégradé.

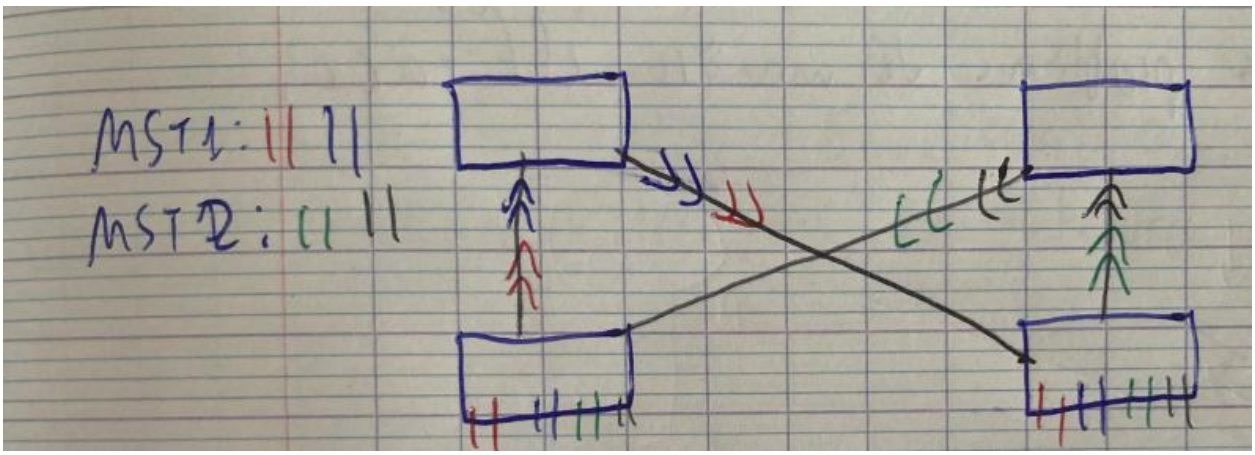


Illustration de l'équilibrage des charges selon MSTP

Synthèse des protocoles Spanning-Tree :

Ainsi, les trois déclinaisons principales du Spanning-Tree ont été étudiées, leur fonctionnement respectif expliqué. L'intérêt de ces études au cas par cas est une comparaison finale afin de déterminer quel protocole, avec quel paramétrage, utiliser dans un milieu professionnel dans le but d'optimiser la redondance, les temps de maintenance et la vitesse de re convergence.

Ce projet s'est ouvert par l'étude du protocole STP classique IEEE 802.1D, il a été possible de conclure que ce protocole n'est plus compatible avec les besoins de performance actuels. En effet, le temps de synchronisation et de re convergence est trop long, plus de cinquante secondes dans certains cas. Néanmoins, ce protocole apporte une solution viable pour contrer la multiplication sans fin du trafic BUM comme illustré page 12.

Le deuxième protocole étudié est RSTP, IEEE 802.1w, il s'agit de l'évolution du protocole STP. L'intérêt principal de ce protocole est sa vitesse de convergence, en effet, ce dernier mobilise un système de synchronisation permettant une réaction extrêmement rapide aux changements.

Enfin, nous nous sommes penchés sur la dernière déclinaison du Spanning-Tree, MSTP, qui permet de limiter l'envoi de BPDUs tout en conservant les caractéristiques de vitesse propre à RSTP. De plus, MSTP permet d'équilibrer le trafic en lui faisant emprunter différents chemins afin d'éviter une surcharge. Le réseau peut également fonctionner dans un mode dégradé avec MSTP en cas de perte de chemin suite à une panne.

Pour conclure, MSTP est le protocole le plus adapté pour des infrastructures en charge d'un volume de trafic important.

De fait, ces réseaux sont généralement composés d'un grand nombre de VLANs rendant la gestion de BPDUs difficile si les instances ne sont pas regroupées pour limiter l'envoi de message de configuration.

Solutions Alternatives :

Comme expliqué, le spanning-tree représente une solution tangible pour gérer les boucles qui se forment dans la topologie. Les protocoles qui l'utilisent ont l'avantage de fonctionner directement, sans besoin de configuration préalable. Néanmoins, la désactivation de liens dans le but de gérer les boucles peut s'avérer préjudiciable pour les performances du réseau, la redondance se voit temporairement affectée et l'équilibrage des charges est compromis.

Il est nécessaire de s'interroger sur la mise en place de solutions alternatives ou combinées avec le spanning-tree dans le but de gérer les soucis de performance tout en évitant les tempêtes Broadcast.

Agrégation de Ports :

La première solution envisageable est l'agrégation de ports, c'est-à-dire le rassemblement de différents liens en un seul lien logique. Cela permet de conserver la capacité de deux liens, le spanning-tree reconnaît ces liens combinés comme un seul lien, ce qui permet d'éviter la mise hors service de segments.

Cette technique d'agrégation de port est idéale pour une utilisation avec l'algorithme Spanning-Tree, le problème de bande passante est partiellement réglé. Malgré tout, ce mécanisme n'est pas idéal et ne résout pas entièrement la problématique ; les liens sont mis hors service dans le but d'éliminer les boucles.

Agrégation de Ports couplée à MSTP :

Afin de démontrer le fonctionnement et l'efficacité de l'agrégation de ports, nous allons mettre en place cette solution dans une topologie dépendant de MSTP. Il s'agit d'une topologie à trois commutateurs, identique à celle utilisée pour la démonstration du protocole *IEEE. 802.1s*, *voir section V.*

Chaque lien trunk entre les Switches sera un *port Channel*, une agrégation de port suivant le protocole *LACP*, *IEEE 802.1AX*. LACP permet de rassembler plusieurs liens physiques en un unique lien logique. Ce lien logique conserve la capacité technique des différents liens physiques qui le composent.

La configuration des agrégations suivant LACP s'effectue comme tel :

SW(config)# *interface Port-channel [identifiant_groupe]*

SW(config-if)# *switchport mode trunk*

SW(config-if)# *switchport trunk allowed vlan 10,20,30,40*

SW(config)# *interface range gx/x – gx/x*

SW(config-if)# *switchport mode trunk*

SW(config-if)# *switchport trunk allowed vlan 10,20,30,40*

SW(config-if)# *channel-group [idenfiant_groupe] mode active*

Les ports de l'agrégation sont rassemblés par Switch sous un identifiant ; un nombre, défini dans la configuration. Il est possible d'inspecter les ports rassemblés sous une agrégation, *voir fig.6.1*.

```
Flags:  D - down        P - bundled in port-channel
        I - stand-alone s - suspended
        H - Hot-standby (LACP only)
        R - Layer3      S - Layer2
        U - in use      f - failed to allocate aggregator

Group  Port-channel  Protocol    Ports
------+-------------+----------+-----------------------------------------------
1      Po1(SU)         LACP      Gi1/0/1(P)  Gi1/0/2(P)
```

Figure 6.1 : Résumé des agrégations de ports par Switch après configuration LACP

La figure précédente montre les interfaces couplées et l'état de l'agrégation, si cette dernière est fonctionnelle ou non.

Ainsi, cette solution permet de conserver les avantages de MSTP comme l'équilibrage des charges ou la réduction du nombre de BPDUs nécessaire au fonctionnement de la topologie. De plus la bande passante du réseau est améliorée grâce à l'agrégation de ports.

L'utilisation de LACP ne permet pas de résoudre intégralement la problématique liée à l'utilisation du spanning-tree mais en atténue les effets sur les performances réseau.

Architecture de LANs :

Les structures réseaux ont vu émerger différentes architectures de LANs ces dernières années, notamment utilisées dans des *Data centres*. Cette section sera consacrée à l'étude de deux de ces structures, la *fabrique* et le *Spine-Leaf*, ces dernières permettent d'assurer la gestion d'une quantité de trafic importante tout en gérant les redondances de Layer 2.

Fabrique :

La fabrique est une architecture de Data centre, composée de trois couches hiérarchiques. Cette organisation est particulièrement efficace pour gérer le trafic Nord-Sud, c'est-à-dire le trafic qui traverse les trois couches, des hôtes dans les VLANs au routeur qui fait sortir le trafic vers des réseaux extérieurs.

La première couche hiérarchique s'appelle la couche de distribution, cette dernière connecte directement les hôtes dans les VLANs. Les Switches qui composent cet étage possèdent un grand nombre de ports destinés à accueillir un nombre aussi important de clients / hôtes.

La deuxième couche est la couche de distribution, qui fait le lien entre le trafic de Layer 2 à Layer 3. Cet étage est généralement composé de Switches de Layer 3 qui se chargent de router le trafic entre les VLANs, là où sont configurées les SVIs.

La dernière couche se nomme couche Cœur, composée exclusivement de routeurs, uniquement responsable de trafic de Layer 3, sert à connecter les différentes couches de distribution dans des LANs importants. Cet étage est utilisé pour sortir du LAN et router le trafic vers l'extérieur.

La *figure 8.1* [5]illustre le concept de Fabrique :

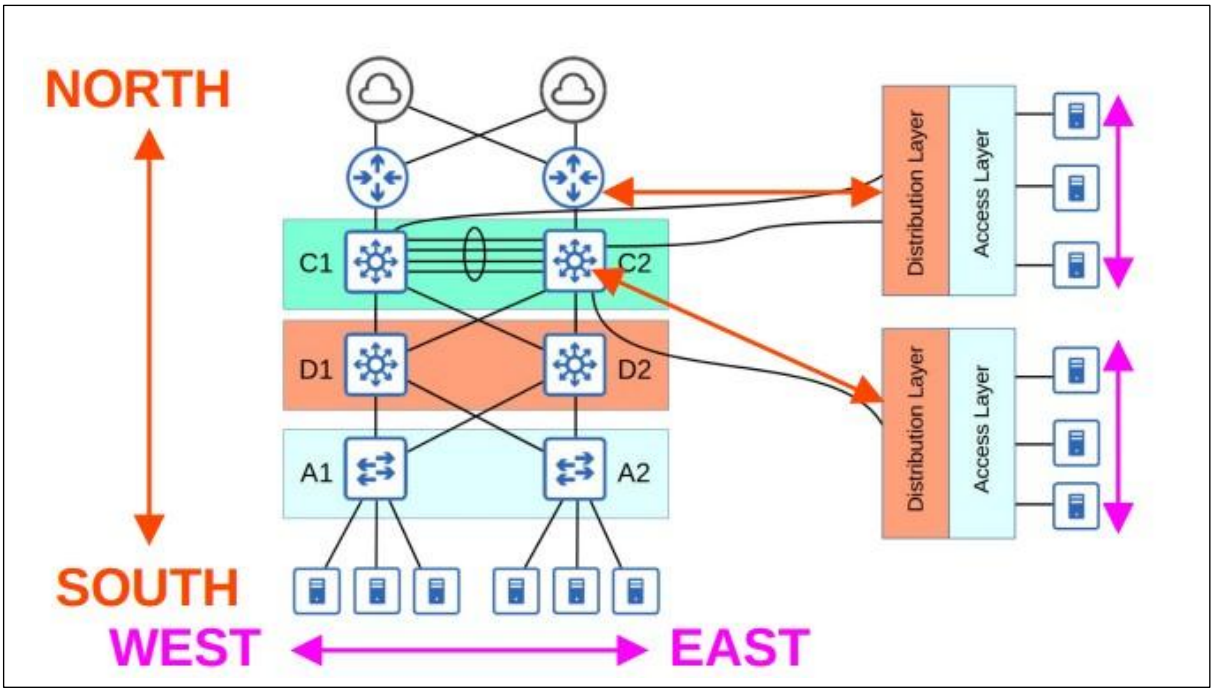


Figure 8.1 : Illustration d'une Fabrique et des couches associées

[5] https://courses.jeremysitlab.com/

Architecture Spine-Leaf :

Cette architecture est principalement utilisée au sein de Data centres et permet de gérer le trafic *Est-Ouest*. Il s'agit de trafic qui ne sort pas du réseau, qui va de la gauche vers la droite et inversement. L'Est-Ouest se traduit majoritairement par l'utilisation de machines virtuelles (VMs) qui émettent des requêtes vers d'autres VMs dans le même Data Centre.

Cette architecture est composée de deux couches, de manière similaire à la fabrique, il existe une couche d'accès et une couche de distribution / agrégation.

La couche d'accès est uniquement composée de Switches de Layer 2 dits *Leaf Switches*, ces commutateurs servent d'accès pour les hôtes et segmentent la topologie au layer 2 avec les VLANs.

La couche de distribution est composée de Switches de Layer 3 dits *Spine Switches*, ces commutateurs sont utilisés pour le routage inter-VLAN (soit le trafic Est-Ouest).

Cette architecture se doit de suivre plusieurs règles :

- Aucun Leaf Switch ne doit être connecté à un autre Leaf Switch
- Aucun Spine Switch ne doit être connecté à un autre Spine Switch
- Chaque Leaf Switch est connecté à tous les Spine Switches
- Aucun hôte ne doit être connecté à un Spine Switch

Ces règles permettent d'obtenir une distance maximale de trois sauts pour qu'un hôte puisse en atteindre un autre, *voir fig. 9.1*.[6]

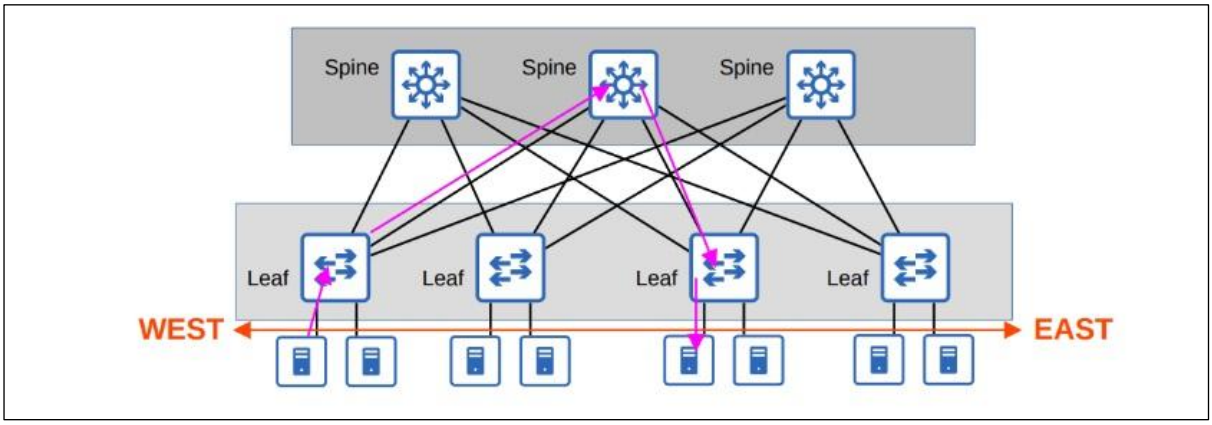


Figure 9.1 : Illustration d'une architecture Spine-Leaf

Comme nous pouvons le constater, la redondance est assurée sans boucle au niveau 2, ce qui permet de conserver tous les liens actifs. Cette architecture représente un progrès considérable dans la gestion des segments de Layer 2, le spanning-tree est toujours utilisé mais ne place aucun lien en attente, la vitesse de transmission de trafic n'est pas dégradée.

[6] https://courses.jeremysitlab.com/

Application Pratique :

Afin d'illustrer au mieux le fonctionnement de cette architecture, ses performances vont être comparées à celles d'un domaine de spanning-tree classique. La topologie est disponible *fig. 9.2*.

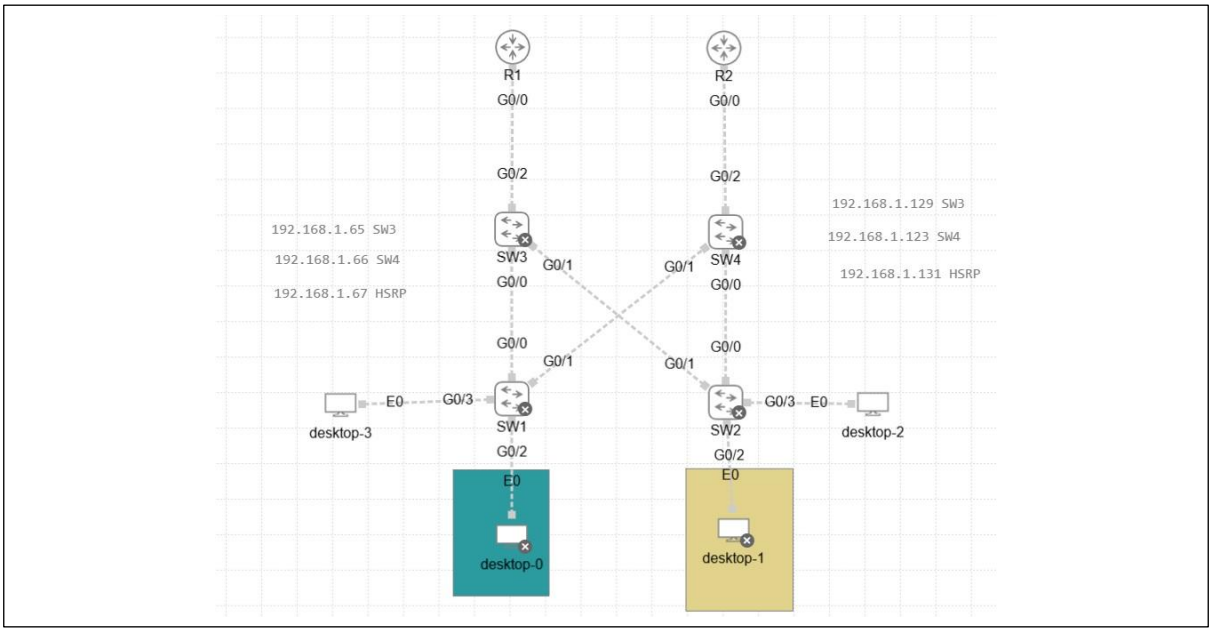


Figure 9.2 : Mise en place d'une architecture Spine-Leaf dans CML

En somme, cette architecture est efficace pour la gestion d'un volume de trafic important et permet de résoudre les problèmes liés au spanning-tree et à la désactivation des liens, comme démontré en laboratoire.

Conclusion Générale :

Ainsi, le Spanning-tree est encore d'actualité dans les architectures modernes, il existe différentes solutions pour optimiser son fonctionnement telles que les architectures Spine-Leaf ou encore Fabrique. Au cours du projet, différentes configurations ont été étudiées afin de déterminer les meilleures solutions vers une optimisation des communications de Layer 2, tout en comparant celles déjà mises en place dans des conditions réelles.
L'intégration de protocoles comme RSTP et MSTP, combinée à des mécanismes tels qu'EtherChannel *via LACP*, permet d'améliorer la résilience et la rapidité de convergence du réseau.

Enfin, les topologies modernes, comme l'architecture Spine-Leaf ou Fabrique, s'inscrivent dans une logique de réduction des points de défaillance et de maximisation des performances, tout en restant compatibles avec les fondements du Spanning-tree.